AWLS: Advantage-Weighted Latent Steering for One-Step Generative Robotic Manipulation Policies

Jiahong Tang[1,2] **Jiaju He**[1,2] **Zhiyuan Yang**[1,2] **Ruyang Han**[1,2] **Haiyue Zhu**[2‡]

[1]Nanyang Technological University, Singapore
[2]Singapore Institute of Manufacturing Technology (SIMTech),
Agency for Science, Technology and Research (A*STAR), Singapore
[‡]Corresponding author

Abstract: Human-collected real-world robotic manipulation demonstrations often contain mixed-quality trajectories, including hesitant, indirect, or inefficient action segments. While diffusion- and flow-based policies can model such multi-modal data, standard behavior cloning (BC) assigns equal likelihood to all demonstrated actions and may therefore reproduce suboptimal modes. Offline RL value estimates offer a natural signal for biasing policies toward higher-value behavioral modes, but directly fine-tuning the generative policy or using critic gradients for policy steering can amplify critic errors and induce prior drift. We propose Advantage-Weighted Latent Steering (AWLS), an offline post-training method for improving frozen one-step generative manipulation policies without updating their decoders. AWLS learns a residual noise shaper, an observation-conditioned residual correction to the policy's latent noise, and trains it with an advantage-weighted action reconstruction objective. The advantages are detached, scale-normalized, and used only as supervised weights on demonstrated actions, avoiding critic-gradient policy updates while preserving fast single-step inference. We further introduce a rollout-free decoder-sensitivity criterion for selecting pretrained checkpoints that remain responsive to latent-noise perturbations. Experiments on simulated and real-world robotic manipulation tasks with clean and mixed-quality demonstrations show that AWLS improves over the frozen base policy while limiting drift from the pretrained behavior prior. Project page: https://awls-corl2026.github.io/.

Keywords: Robot Manipulation, Generative Policies, Latent Steering

1 Introduction

Generative policies based on diffusion and flow matching [1, 2, 3] have become a standard approach to imitation learning for robotic manipulation. They are typically trained by behavior cloning (BC), which fits the policy to human-collected demonstrations and therefore inherits their quality. Yet even successful demonstrations are rarely uniformly optimal. Some trajectories are smooth and efficient, while others are hesitant, indirect, or unnecessarily long [4, 5]. Standard BC does not distinguish between these quality levels, so it reproduces hesitant behavior as faithfully as efficient behavior (Figure 1). Can a policy trained on such data instead be steered toward the better behavior in those demonstrations?

Offline reinforcement learning (RL) offers an answer: value estimates learned from the same demonstrations can identify higher-advantage demonstrated actions in their respective contexts. The question then becomes how to inject this signal into a pretrained generative policy. Guiding the sam-

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

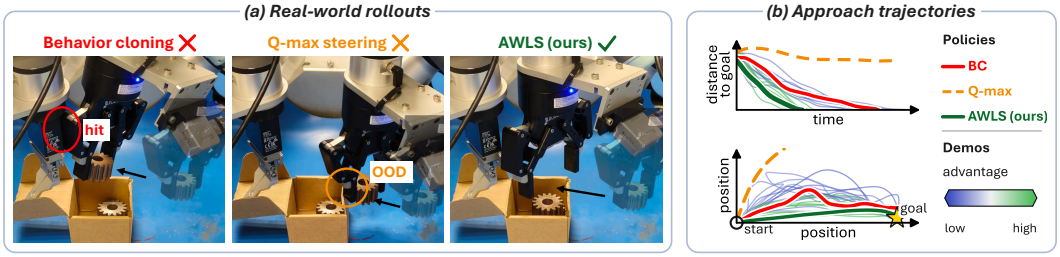


Figure 1: **(a)** Real-world rollouts of the same grasp: behavior cloning reproduces a low-quality mode that collides with the box, Q-maximization steering chases high-Q actions outside the demonstration support (OOD), and AWLS reshapes the latent noise of the frozen decoder toward the demonstrated mode that reaches the goal. **(b)** Approach trajectories as distance to goal over time (top) and start-to-goal paths (bottom): thin lines are demonstrations colored by advantage (blue low, green high), and thick curves are policy rollouts. AWLS reproduces the more efficient demonstrated trajectories with fewer detours, whereas BC follows a slower one and Q-max steering leaves the demonstrated region.

pling process with the value signal [6, 7, 8] keeps the critic in the loop at inference time, and fine-tuning the decoder with value gradients can amplify offline critic errors while eroding the pretrained prior [9, 10]. This makes frozen-decoder latent steering an appealing interface: the pretrained decoder is kept fixed, and improvement is attempted by reshaping the latent noise. Yet this interface exposes two hidden failure modes. First, critic-gradient steering in latent space [11, 12, 13, 14] can still chase extrapolated value artifacts rather than demonstrated high-value behavior. Second, latent steering is only effective if the frozen decoder remains sensitive to its latent noise, a property that behavior cloning may weaken during training.

We propose Advantage-Weighted Latent Steering (AWLS) to mitigate both failure modes simultaneously (Figure 1(a)). AWLS freezes the pretrained decoder and learns only a residual noise shaper that applies observation-conditioned corrections to the latent-noise input. These corrections are initialized near zero to bias post-training toward the original latent prior. Critically, the offline critic never provides a gradient to the policy: advantages estimated via implicit Q-learning (IQL) [15] are detached and enter training only as scalar weights on demonstrated actions [16]—critic errors may change which demonstrated actions are emphasized without directly providing gradients toward potentially unsupported actions. Two additional mechanisms help make this interface practical across tasks: scale-normalized advantage weighting bounds weight concentration independently of reward magnitude, and a rollout-free decoder-sensitivity diagnostic selects frozen checkpoints that remain responsive to latent-noise perturbations. These choices map directly to the two failure modes above: detached advantage weights remove critic-gradient policy updates, while decoder-sensitivity checkpoint selection addresses sensitivity degradation during BC training.

Our contributions are: **(i)** we identify and address two reliability issues in frozen latent steering for one-step generative manipulation policies, using detached advantage weights to address critic-gradient failure modes and decoder-sensitivity checkpointing to address sensitivity degradation; **(ii)** we propose AWLS, a residual latent-noise shaper trained by advantage-weighted reconstruction with scale-normalized weights while keeping the decoder frozen; **(iii)** we evaluate AWLS on simulation benchmarks, controlled mixed-quality demonstrations, and real-world robot tasks.

2 Related Work

Generative policy learning. Diffusion and flow policies learn expressive action distributions for manipulation [17, 18], using RGB and depth perception [1, 2, 19] to condition action generation. Few- and one-step variants reduce sampling cost for fast closed-loop control [20, 21]. Building on these policy classes, offline RL uses learned value functions to guide policy training and action selection [22, 23, 24]. Pretrained policies can also be adapted through offline-to-online RL, policy-

gradient updates, or generative-model optimization [10, 25, 9, 26, 27, 28]. In offline settings, value-gradient updates can exploit critic extrapolation errors and move the policy away from its behavior-cloning prior [29]. Behavior-cloning regularization can reduce this drift, but the policy update still depends on critic gradients.

Frozen-decoder post-training. A complementary line keeps the pretrained decoder fixed. Inference-time guidance steers multi-step denoising with value functions, return conditioning, or dynamics guidance [6, 7, 8, 30], but needs an iterative sampler that one-step generators do not expose. Noise-space steering instead adapts the latent noise of a frozen decoder through critic gradients, rollout search, or external feedback [11, 12, 13, 14]; critic-gradient variants still depend on the learned value landscape, and search- and feedback-based variants need additional evaluation signals. Latent-action offline RL [31, 32] trains a VAE-style latent policy with value gradients or advantage weights, and learned noise priors [33] guide an iterative diffusion sampler. AWLS differs from all of these in how it uses the latent interface. Following advantage-weighted imitation [16, 34, 35], offline advantages estimated with IQL [15] serve only as detached weights for supervised reconstruction of demonstrated actions. AWLS therefore trains no new generative model, keeps single-step inference, and needs neither value gradients nor critic-defined targets.

3 Method

AWLS is a decoder-preserving post-training operator for one-step generative robot policies. It combines three components: the frozen decoder supplies the action prior, an offline critic supplies only detached preferences over demonstrated actions, and a residual noise shaper T_ϕ, the only trainable module, acts solely through the decoder's latent noise. Figure 2 contrasts the two stages: the top row shows standard behavior cloning, and the bottom row shows AWLS post-training through a frozen decoder. This section first states the problem through the one-step interface, then derives the advantage-weighted latent objective, the scale-normalized weighting rule, and the checkpoint criterion needed for the interface to remain effective.

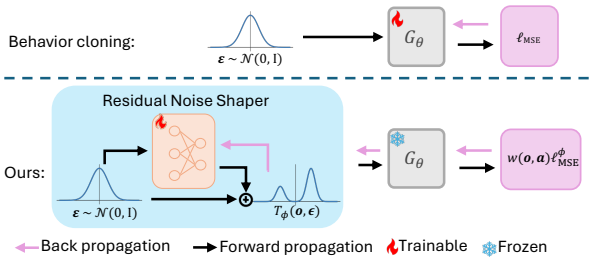


Figure 2: **Overview of AWLS.** *Top:* the one-step decoder G_θ is pretrained by behavior cloning. *Bottom:* with G_θ frozen, T_ϕ reshapes the latent noise via advantage-weighted reconstruction using detached, normalized offline advantages.

3.1 Problem Statement: Frozen One-Step Decoders

We consider offline post-training for a pretrained one-step generative policy [36, 37, 38, 39] that maps the current observation $\boldsymbol{o}$ and Gaussian latent noise $\boldsymbol{z}$ to an action chunk $\boldsymbol{a}$:

$$\boldsymbol{a} = G_\theta(\boldsymbol{o}, \boldsymbol{z}), \qquad \boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{1}$$

where *one-step* means that one action chunk is produced by a single evaluation of G_θ. Multi-step diffusion or flow-matching samplers are also differentiable, but optimizing them through an action-space objective requires unrolling the sampling chain; Eq. (1) instead exposes a direct, unroll-free gradient path from the action-space loss back to the latent noise. The decoder G_θ is *frozen* throughout post-training.

We are given an offline dataset $\mathcal{D}$ of mixed-quality demonstrations, each providing transitions $(\boldsymbol{o}, \boldsymbol{a}, r, \boldsymbol{o}')$; the critic below uses full transitions, whereas the policy loss uses only the observation–action pairs $(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}})$. The reward is the same binary task-success reward used for evaluation: it is emitted when the task first reaches its success condition, and the episode terminates on success,

so with discount $\gamma < 1$ earlier success yields a larger return. These rewards are used only to fit the offline critic of Section 3.2; policy post-training itself remains anchored to demonstrated actions. AWLS learns only a noise shaper T_ϕ that maps base noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to the decoder input $\boldsymbol{z} = T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$, so that $\boldsymbol{a} = G_\theta(\boldsymbol{o}, T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}))$ preferentially reconstructs higher-quality demonstrated actions. For a demonstrated pair $(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}})$ we define the noise-averaged mean squared error (MSE)

$$\ell^{\phi}_{\text{MSE}}(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})} \left[\left\| G_\theta\big(\boldsymbol{o}, T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})\big) - \boldsymbol{a}_{\text{demo}} \right\|_2^2 \right], \tag{2}$$

which depends on ϕ only through T_ϕ. The identity shaper $T_\phi = \text{id}$ recovers the pretrained BC policy; Eq. (2) defines the action reconstruction loss used during post-training.

3.2 Advantage-Weighted Latent-Steering Objective

Motivated by advantage-weighted regression (AWR) [16], AWLS weights the reconstruction loss by exponentiated advantages estimated offline via IQL [15] (Appendix B). A pessimistic critic ensemble gives $Q_{\text{pess}}(\boldsymbol{o}, \boldsymbol{a}) = \min_k Q_{\psi_k}(\boldsymbol{o}, \boldsymbol{a})$, an expectile-trained value function V_ν provides the baseline, and the advantage is the detached difference

$$A(\boldsymbol{o}, \boldsymbol{a}) = \text{sg}\big[Q_{\text{pess}}(\boldsymbol{o}, \boldsymbol{a}) - V_\nu(\boldsymbol{o})\big], \tag{3}$$

where $\text{sg}[\cdot]$ denotes stop-gradient. The raw AWR weight has the form $w_{\text{raw}}(\boldsymbol{o}, \boldsymbol{a}) = \exp(\beta A(\boldsymbol{o}, \boldsymbol{a}))$. Likelihood-based AWR fits the reweighted behavior distribution $\pi^*(\boldsymbol{a} \mid \boldsymbol{o}) \propto \pi_{\mathcal{D}}(\boldsymbol{a} \mid \boldsymbol{o}) \exp(\beta A(\boldsymbol{o}, \boldsymbol{a}))$, which assigns greater probability to higher-advantage demonstrated actions. AWLS adopts this weighting principle, but the marginal log-density of its composed policy is not directly tractable: even for dimension-matched invertible maps, evaluation requires an inverse mapping and the Jacobian log-determinants of both G_θ and T_ϕ [40]. We therefore use the action reconstruction loss of Eq. (2) as an AWR-inspired supervised surrogate:

$$\mathcal{L}_{\text{AWLS}}(\phi) = \mathbb{E}_{(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}}) \sim \mathcal{D}} \Big[w(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}})\ \ell^{\phi}_{\text{MSE}}(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}}) \Big], \tag{4}$$

where the weight $w(\boldsymbol{o}, \boldsymbol{a}) = \exp(\beta\, \tilde{A}(\boldsymbol{o}, \boldsymbol{a}))$ uses a scale-normalized advantage $\tilde{A} \in [0, 1]$ defined by Eq. (6) in Section 3.3. The critic affects training only through the detached weight $w(\boldsymbol{o}, \boldsymbol{a}_{\text{demo}})$: advantages, batch statistics, and weights carry no gradient, so AWLS neither backpropagates through Q_{pess} nor uses an action-direction gradient $\nabla_{\boldsymbol{a}} Q_{\text{pess}}$, and the only gradient flows from the MSE loss through the frozen decoder into T_ϕ.

This objective anchors the training targets to demonstrated actions, but does not establish equivalence to likelihood-based AWR or guarantee that generated actions remain within the demonstration support. Because independent noise samples are regressed toward demonstrated targets, reconstruction can also reduce conditional action diversity (Appendix C). To bias adaptation toward the pretrained latent input, AWLS uses the residual form

$$T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon} + \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}), \tag{5}$$

where Δ_ϕ is a small MLP conditioned on the observation encoding and $\boldsymbol{\epsilon}$ (Appendix F.2), with its output initialized near zero, so post-training starts near the identity shaper. The residual form supplies an initialization and optimization bias; its behavior is assessed through residual displacement, local Jacobian diagnostics, and action-diversity comparisons. If $\|\partial \Delta_\phi / \partial \boldsymbol{\epsilon}\|_{\text{op}} \le L_v < 1$ at a state–noise pair, the shaper Jacobian is nonsingular and its singular values lie in $[1 - L_v, 1 + L_v]$ (Prop. 2). This controls local geometry, rather than the MSE–likelihood discrepancy. The condition is not enforced during training; measurements at the evaluated checkpoints show modest corrections and nonsingular local Jacobians, while the residual shaper retains more action diversity than the tested direct and Gaussian alternatives (Appendices C and F.2).

3.3 Scale-Normalized Advantage Weighting

A practical fragility of the raw AWR weight $w_{\text{raw}} = \exp(\beta A)$ is that the effective sharpness $\beta\, A$ depends on the magnitude of the advantage, which varies widely across tasks: different horizon

lengths, reward densities, and return magnitudes produce different advantage ranges even under the same reward convention. A fixed β therefore has no consistent meaning: on a task with small advantages the weights become nearly uniform, while on a task with large advantages they overflow `float32` or collapse the effective sample size (ESS) of the batch toward 1, concentrating all weight on a single sample. To decouple the inverse temperature β from this task-specific advantage scale, we normalize the advantage to the unit interval before exponentiation. Given a minibatch $\mathcal{B}$, define

$$\tilde{A}(\boldsymbol{o}, \boldsymbol{a}) = \frac{A(\boldsymbol{o}, \boldsymbol{a}) - A^{\mathcal{B}}_{\min}}{A^{\mathcal{B}}_{\max} - A^{\mathcal{B}}_{\min} + \varepsilon}, \qquad \tilde{A} \in [0, 1], \tag{6}$$

where $A^{\mathcal{B}}_{\min}$ and $A^{\mathcal{B}}_{\max}$ are the batch extrema and $\varepsilon > 0$ is a small constant for numerical stability. Note that both $\tilde{A}$ and w depend on the current minibatch; we suppress this dependence for brevity. Substituting this normalized advantage into the weight w of Eq. (4) bounds the weight: since $\tilde{A} \in [0, 1]$, $w \in [1, e^{\beta}]$ regardless of the reward scale, advantage range, or critic magnitude. In contrast, the raw AWR weight w_{raw} produces a within-batch weight ratio of $e^{\beta(A^{\mathcal{B}}_{\max} - A^{\mathcal{B}}_{\min})}$, which grows exponentially with the advantage range.

Equivalently, per-batch normalization applies an adaptive inverse temperature $\beta_t = \beta/(\Delta_t + \varepsilon)$ to min-shifted advantages, where $\Delta_t = A^{\mathcal{B}_t}_{\max} - A^{\mathcal{B}_t}_{\min}$. This automatically adapts to the evolving advantage magnitude at each training step. We use $\beta = 3$ for all tasks (Section 4.4).

Proposition 1 (Bounded weight concentration)**.** *For any minibatch of size n, the AWLS weights satisfy $w_{\max}/w_{\min} \le R$ with $R = e^{\beta}$, and the effective sample size is lower-bounded by* $\text{ESS} = (\sum_i w_i)^2 / \sum_i w_i^2 \ge 4nR/(1+R)^2$*. For $\beta = 3$ and $n = 256$, this gives* $\text{ESS} \ge 46.3$*. The bound holds at every training step regardless of the critic state.*

The bound controls worst-case weight concentration only; it does not guarantee that the critic ranking is accurate or informative. The proof is in Appendix D.

3.4 Frozen Base Checkpoint Selection

Latent steering can change the decoded action only through the local noise–action Jacobian $J_\theta(\boldsymbol{o}, \boldsymbol{z}) = \partial G_\theta(\boldsymbol{o}, \boldsymbol{z})/\partial \boldsymbol{z}$. For a sampled $\boldsymbol{\epsilon}$ with $\boldsymbol{z} = T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$ and $\hat{\boldsymbol{a}} = G_\theta(\boldsymbol{o}, \boldsymbol{z})$, the gradient of the squared error $\ell = \|\hat{\boldsymbol{a}} - \boldsymbol{a}_{\text{demo}}\|_2^2$ that reaches the shaper is the action-space gradient pulled back through the decoder, $\nabla_\phi \ell = (\partial T_\phi/\partial \phi)^\top J_\theta(\boldsymbol{o}, \boldsymbol{z})^\top \nabla_{\hat{\boldsymbol{a}}} \ell$. When J_θ has low gain or is concentrated in a few directions, this gradient provides little leverage for a latent-only update. We observe that such loss of latent sensitivity can occur late in BC training even after task performance has plateaued (Appendix E), so the final BC checkpoint is not necessarily the best frozen base for latent steering. We therefore score the checkpoints $\{\theta_k\}$ saved during BC training with two rollout-free diagnostics computed on held-out demonstrations. *Best-of-M reconstruction error* L_{BoM} proxies BC quality by the best action-space match over M latent-noise samples. *Directional sensitivity* S_J is a finite-difference estimate of the gain of J_θ: the decoded action change under small random latent-noise perturbations. Among checkpoints whose BC quality is within a tolerance δ of the final checkpoint ($\delta = 0.2$ throughout), we select the one with the highest decoder sensitivity:

$$\theta^* = \arg\max_{\theta_k \in \mathcal{C}_\delta} S_J(\theta_k), \qquad \mathcal{C}_\delta = \big\{\theta_k : L_{\text{BoM}}(\theta_k) \le (1+\delta)\, L_{\text{BoM}}(\theta_{\text{final}})\big\}. \tag{7}$$

Formal definitions and per-task diagnostics are provided in Appendix E.

Optimization and deployment. Each post-training step samples a demonstration minibatch, computes detached IQL advantages and their normalized weights, draws one independent base-noise sample per pair as a single-sample estimate of Eq. (2), and takes a gradient step on Eq. (4) with respect to ϕ only. At deployment the critic and advantage computation are removed: AWLS samples $\boldsymbol{\epsilon}$, evaluates $T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$, and performs one decoder evaluation, so inference remains a single generative step without iterative denoising or candidate selection.

Paradigm	Method	Denoising Steps	Adroit (3 tasks)	DexArt (4 tasks)	RoboTwin (5 tasks)	Average
Behavior Cloning	DP3	10	67.5±3.3	50.3±1.2	45.0±3.8	52.4±2.8
	FlowPolicy	1	71.3±2.0	50.4±2.2	48.3±4.2	54.8±3.0
	MeanFlow	1	72.1±4.2	54.7±2.2	53.2±4.6	58.4±3.7
Q-maximization	DSRL	10	72.5±2.5	56.4±3.7	55.8±4.8	60.2±3.9
	FQL	1	73.6±2.6	57.4±2.4	56.1±4.9	60.9±3.5
Ours	FP-AWLS	1	74.8±2.6	58.5±2.1	57.6±4.5	62.2±3.2
	MF-AWLS	1	**76.1±3.5**	**62.2±1.3**	**61.6±3.7**	**65.4±2.9**

Table 1: Cross-task average success rate (%) by benchmark. Each entry averages the per-task success rates of the benchmark (means over three random seeds); the ± value is the average of the per-task standard deviations (per-task results in Appendix G).

4 Experiments

4.1 Simulation Experiments

Setup. We evaluate AWLS on Adroit [41], DexArt [42], and RoboTwin [43], covering 12 simulation tasks with single-hand dexterity, articulated-object interaction, and long-horizon bimanual manipulation. Demonstrations are generated by simulation RL expert policies. Although these experts achieve high success, their rollouts still vary in trajectory length, smoothness, contact quality, and hesitation. We use simulation not as a replacement for human teleoperation, but as a controlled testbed for isolating the within-success quality variation that makes real-world teleoperated data difficult: even successful rollouts can differ in efficiency, contact timing, and recovery behavior. We use 10 demonstrations per Adroit task, 100 per DexArt task, and 50 per RoboTwin task. All simulation experiments are run on a single NVIDIA RTX 3090 GPU.

Baselines. We compare with behavior-cloning policies DP3 [2], FlowPolicy [44], and MeanFlow [37], as well as Q-maximization methods FQL [24] and DSRL [11]. FQL and DSRL are Q-maximization post-training methods built on FlowPolicy and DP3 backbones, respectively. FP-AWLS applies our post-training procedure to a FlowPolicy-style one-step decoder, while MF-AWLS applies it to a MeanFlow decoder. All methods use the same offline demonstrations, and each AWLS variant keeps the corresponding behavior-cloning decoder frozen at evaluation time.

Post-training setup. We first train each one-step decoder by behavior cloning, for 3000 epochs on Adroit and DexArt and 1000 epochs on RoboTwin (per-benchmark settings in Appendix F.1). FP-AWLS and FQL start from the same pretrained FlowPolicy decoder, so their comparison isolates the post-training procedure. Both AWLS variants select their frozen base from the BC checkpoints with the rollout-free S_J criterion of Section 3.4 (Eq. (7)); AWLS then freezes the selected decoder and updates only the residual noise shaper T_ϕ. The offline critic is trained on the same demonstrations and supplies detached advantage weights; critic gradients are never backpropagated into the policy decoder. Each method is trained across three random seeds, and every reported simulation success rate is the mean over 20 evaluation episodes per task per seed.

Results. Table 1 shows that AWLS consistently improves the frozen one-step base policies across the three simulation benchmark families. Both AWLS variants improve over their respective base policies on every benchmark family; per-task breakdowns in Appendix G confirm the trend across all 12 tasks. Under the matched FlowPolicy backbone, the FP-AWLS margin over FQL is modest (62.2 vs. 60.9 on average); FQL's behavior-regularization coefficient is tuned per task following its official implementation, whereas AWLS uses a single fixed $\beta = 3$ for all tasks. Our claim is therefore competitive performance under a simpler, shared-configuration frozen-decoder post-training protocol rather than a large margin over tuned Q-maximization baselines.

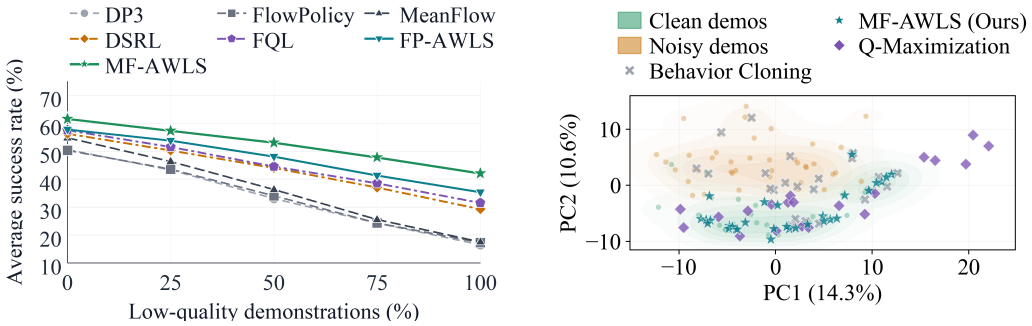


Figure 3: **Left:** Average success rate on four DexArt tasks as the noisy-demonstration fraction increases. **Right:** PCA projection of flattened Bucket action trajectories with 50 clean and 50 noisy demonstrations.

4.2 Mixed-Quality Demonstrations

We next evaluate robustness to mixed-quality demonstrations on DexArt. Starting from the same RL expert demonstrations used in simulation, we inject temporally correlated Ornstein–Uhlenbeck (OU) action noise into expert rollouts to construct a low-quality demonstration set. The noise level is chosen so that the expert rollout success rate drops from near 100% to below 20%, while still producing trajectories that remain close to the original task structure; details are provided in the appendix. We then mix clean expert demonstrations and noisy expert-derived demonstrations at different ratios and train each method on the resulting offline dataset. We use the same success-rate evaluation protocol as in Section 4.1.

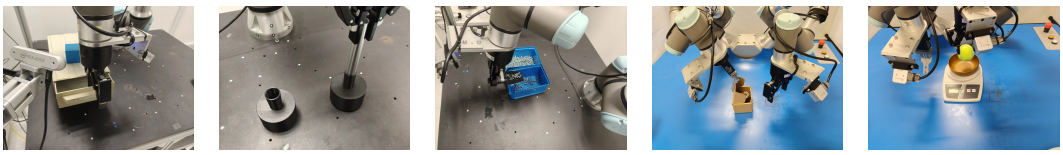
As the noisy-demonstration fraction increases (Figure 3, left), all methods degrade, but AWLS maintains higher success than the behavior-cloning and Q-maximization baselines. The trajectory PCA (Figure 3, right) further shows how the methods use the mixed dataset. Behavior cloning spreads generated trajectories across both clean and noisy demonstration modes because it treats all demonstrations uniformly. AWLS instead concentrates its trajectories near the clean-demonstration cluster in this projection, consistent with a preference for higher-quality demonstrated behavior in the mixed dataset. Q-maximization also shifts many trajectories toward the clean mode, but a nontrivial fraction lies outside both demonstration clusters in this projection, consistent with critic-gradient steering inducing out-of-distribution drift.

4.3 Real-World Experiments

We further evaluate AWLS on real robots to test whether the same offline post-training recipe transfers to physical manipulation. The simulation studies isolate the post-training mechanism under controlled demonstrations; the real-world experiments test the intended setting of AWLS, where policies are trained from human teleoperated demonstrations with natural pauses, corrections, and contact-timing variation.

Setup. For real-world experiments, the single-arm tasks use a UR5e manipulator, whereas the dual-arm tasks use two UR3e manipulators. The perception setup combines one third-view Intel RealSense D455 camera with wrist-mounted Intel RealSense D405 cameras on each arm, providing a shared scene view together with close-range hand-centric observations. Policies consume these two RGB streams plus proprioception and output end-effector pose-and-gripper commands in action chunks at 10 Hz; the offline critic uses the same binary task-success reward as the evaluation criterion. Hardware, observation encoding, and evaluation details are given in Appendix I.

Tasks and demonstrations. Figure 4 shows five real-world tasks: three single-arm tasks and two bimanual tasks. All methods are trained from the same offline teleoperated demonstrations: each task uses 70 demonstrations collected by SpaceMouse teleoperation, with commands expressed in



Drawer-Cube | Tube Insertion | Screw Scooping | Gear Packing | Ball Weighing

Figure 4: Real-world manipulation tasks. The first three are single-arm tasks: **Drawer-Cube** opens a drawer, places a cube, and closes the drawer; **Tube Insertion** removes a jar and inserts a tube; **Screw Scooping** scoops screws and pours them into a target container. The last two are dual-arm tasks: **Gear Packing** opens a box, places a gear, and closes the box; **Ball Weighing** presses the scale button with one arm while placing the ball on the scale with the other arm.

Method	Drawer-Cube	Tube Ins.	Screw Scoop.	Gear Pack.	Ball Weigh.	Total
MeanFlow	12/20	11/20	13/20	4/20	5/20	45/100
FQL	13/20	14/20	15/20	5/20	4/20	51/100
DSRL	12/20	15/20	13/20	6/20	4/20	50/100
Ours	**15/20**	**17/20**	**16/20**	**9/20**	**8/20**	**65/100**

Table 2: Real-world success counts over 20 hardware rollouts per task. MeanFlow is the frozen behavior-cloning base used by MF-AWLS.

the robot base frame. Even when demonstrations are successful, they vary in speed, contact timing, pauses, corrections, and final alignment, creating the mixed-quality conditions that advantage weighting is designed to address.

Training and evaluation. We compare AWLS with the frozen MeanFlow behavior-cloning base, FQL, and DSRL using the same offline data. For real-world experiments, we use the MF-AWLS variant: AWLS freezes the MeanFlow behavior-cloning decoder and trains only the residual noise shaper T_ϕ. No real-world rollouts are used during post-training. Each policy is evaluated for 20 hardware rollouts per task from randomized but feasible initial configurations. A rollout is successful only if every task stage is completed without human intervention.

Results. Table 2 shows that AWLS improves over the frozen MeanFlow base and obtains higher success counts than FQL and DSRL on all five real-world tasks, on both single-arm and bimanual tasks. With 20 rollouts per task the per-task intervals are wide; in a task-stratified test with Holm correction, only the aggregate gain over the frozen MeanFlow base ($65/100$ vs. $45/100$) is statistically significant, while the margins over FQL and DSRL are suggestive (Appendix I). These results support the use of detached advantage weighting to improve policies trained from successful human demonstrations while keeping the pretrained decoder fixed.

4.4 Ablation Studies

We ablate the four design choices of AWLS on the four DexArt tasks with clean demonstrations, using a MeanFlow backbone. Table 3 reports the three discrete comparisons (decoder adaptation, noise-shaper parameterization, and frozen-base checkpoint selection); the continuous advantage-temperature sweep is reported in Appendix G.2 (Figure 7).

Frozen vs. unfrozen decoder. In the decoder-adaptation block, keeping the decoder frozen and steering only the latent noise performs best among the compared variants. Unfreezing decoder parameters reduces the gain, and unfreezing both encoder and decoder performs worse than the BC prior in this setting. The Q-gradient steering ablation gives no improvement over BC, suggesting that the observed gain comes from the advantage-weighted supervised interface rather than latent steering alone.

Advantage temperature β. Appendix G.2 (Figure 7) sweeps the inverse temperature. With the scale-normalized advantage $\tilde{A} \in [0, 1]$, performance is relatively stable across the tested β values,

so we use $\beta = 3$ without task-specific retuning. With the raw advantage $A(\boldsymbol{o}, \boldsymbol{a})$, performance is lower and degrades as β grows, consistent with the unbounded advantage scale over-concentrating the weights (Prop. 1).

Frozen-base checkpoint selection. Selecting the frozen base by our rollout-free S_J criterion improves over selecting the best-BC-score checkpoint in the checkpoint sweep (Table 3(c)); both rows apply the same AWLS post-training, so the difference isolates the choice of frozen base. This suggests that the highest-BC checkpoint is not always the most steerable base, and that decoder sensitivity is a useful selection signal.

	DexArt SR
MF-AWLS (ours)	**62.2**
(a) Decoder adaptation	
BC (no post-training)	-6.7
Q-gradient steering	-6.7
Unfreeze decoder	-4.0
Unfreeze enc. + dec.	-9.7
(b) Noise shaper	
Direct f_ϕ	-2.5
(c) Checkpoint	
BC-best	-8.2
(d) Advantage weighting	
Uniform weights	-6.3
Shuffled advantages	-6.9
Top-50% filtering	-2.0

Table 3: DexArt ablations (MeanFlow, clean demonstrations); compact ΔSR values are relative to MF-AWLS (62.2). Variants were trained in separate batches under the same protocol.

Isolating advantage weighting. Block (d) keeps the selected frozen base and the shaper fixed and varies only the weights (Appendix G.1). Training the same shaper with uniform weights or with shuffled advantages falls 6.3 and 6.9 points below AWLS, back to the level of the BC prior, so the gain requires the learned advantage signal rather than the shaper or the checkpoint alone. Hard top-50% advantage filtering recovers most of the gain (-2.0) but remains below soft reweighting, which retains all demonstrations and is less sensitive to imperfect critic rankings.

Residual vs. direct noise shaper. The residual parameterization outperforms the direct map on clean DexArt (Table 3(b)), suggesting that preserving an identity-centered latent interface is beneficial in practice; the corresponding local geometry and retained action diversity are examined in Appendix C.

5 Conclusion

We propose AWLS, a decoder-preserving post-training method for one-step generative robotic manipulation policies trained from mixed-quality demonstrations. AWLS freezes the pretrained decoder and learns a residual latent-noise shaper with detached, scale-normalized advantage weights, using the critic only to emphasize higher-quality demonstrated actions rather than to directly optimize actions. Across simulation, controlled mixed-quality data, and real-world manipulation tasks, AWLS improves over the frozen base policy and value-maximization baselines while preserving fast single-step inference and limiting drift from the behavior-cloning prior.

6 Limitation

AWLS is intended as a conservative post-training method: it can emphasize better behaviors already present in the offline demonstrations and reachable through the frozen decoder, but it is not designed to synthesize entirely new skills absent from the dataset. Its performance also depends on the offline critic providing a useful relative ranking of demonstrated actions. While detached and scale-normalized weighting reduces sensitivity to critic scale and avoids critic-gradient policy updates, extending AWLS to broader task families, more diverse visual settings, and less clearly ranked datasets remains an important direction for future work.

Acknowledgments

This research is supported by National Robotics Programme (NRP) 2.0 funding initiatives "Domain-specific Robotics Foundation Models for Manufacturing (DS-RFM)" and "Data Engine Minimum Viable Product (DEM)".

References

[1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.

[2] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Robotics: Science and Systems*, 2024.

[3] R. Zhao, W. Wang, Y. Ma, X. Li, F. E. H. Tay, M. H. Ang, Jr., and H. Zhu. FD-VLA: Force-distilled vision-language-action model for contact-rich manipulation. In *IEEE International Conference on Robotics and Automation*, 2026.

[4] D. S. Brown, W. Goo, and S. Niekum. Better-than-demonstrator imitation learning via automatically-ranked demonstrations. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 330–359. PMLR, 2020.

[5] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning*, 2021.

[6] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022.

[7] A. Ajay, Y. Du, A. Gupta, J. Tenenbaum, T. Jaakkola, and P. Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations*, 2023.

[8] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning*, 2024.

[9] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations*, 2025.

[10] M. Sobol Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar. Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone. In *International Conference on Learning Representations*, 2025.

[11] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[12] H. Im, A. Kolobov, J. Fu, and Y. Lee. Latent policy steering through one-step flow policies. *arXiv preprint arXiv:2603.05296*, 2026.

[13] O. Patil, O. Biza, T. Weng, K. Schmeckpeper, W. Thomason, X. Zhang, R. Walters, N. Gopalan, S. Castro, and E. Rosen. You've got a golden ticket: Improving generative robot policies with a single noise vector. *arXiv preprint arXiv:2603.15757*, 2026.

[14] J. Lu, X. Qin, Y. Jiang, K. Wang, C. Zhang, B. Liang, J. Yang, M. Xu, and L. Zhao. Unified noise steering for efficient human-guided VLA adaptation. *arXiv preprint arXiv:2605.10821*, 2026.

[15] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022.

[16] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.

[17] W. Wang, R. Zhao, T. J. Teo, M. H. Ang, Jr., and H. Zhu. Task-guided and object-centric conditioning for effective and adaptive diffusion policy. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 16746–16753, 2025. doi:10.1109/IROS60139.2025.11247520.

[18] W. Wang, R. Zhao, T. J. Teo, M. H. Ang, Jr., and H. Zhu. From dream to action: Hierarchical policy learning with 3D world imagination for robotic manipulation. In *IEEE International Conference on Robotics and Automation*, pages 1155–1162, 2026.

[19] Z. Yang, A. Cheng, H. Zhu, T. Li, P. Y. Tao, and K. Mao. HFD-Teacher: High-frequency depth distillation from depth foundation models for enhanced depth completion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8994–9003, October 2025.

[20] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024.

[21] Z. Wang, Z. Li, A. Mandlekar, Z. Xu, J. Fan, Y. Narang, L. Fan, Y. Zhu, Y. Balaji, M. Zhou, M.-Y. Liu, and Y. Zeng. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. In *International Conference on Machine Learning*, 2025.

[22] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023.

[23] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.

[24] S. Park, Q. Li, and S. Levine. Flow Q-learning. In *International Conference on Machine Learning*, 2025.

[25] J. Zhang, R. Xue, Y. Niu, Y. Chen, J. Yang, H. Li, and Y. Liu. Revisiting generative policies: A simpler reinforcement learning algorithmic perspective. In *International Conference on Learning Representations*, 2025.

[26] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa. Flow matching policy gradients. In *International Conference on Learning Representations*, 2026.

[27] C. Domingo-Enrich, M. Drozdzal, B. Karrer, and R. T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations*, 2025. Spotlight.

[28] K. Zheng, H. Chen, H. Ye, H. Wang, Q. Zhang, K. Jiang, H. Su, S. Ermon, J. Zhu, and M.-Y. Liu. DiffusionNFT: Online diffusion reinforcement with forward process. In *International Conference on Learning Representations*, 2026. Oral.

[29] A. Kumar, A. Zhou, G. Tucker, and S. Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.

[30] M. Du and S. Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems*, 2025.

[31] W. Zhou, S. Bajracharya, and D. Held. PLAS: Latent action space for offline reinforcement learning. In *Proceedings of the Conference on Robot Learning*, 2021.

[32] X. Chen, A. Ghadirzadeh, T. Yu, J. Wang, A. Y. Gao, W. Li, B. Liang, C. Finn, and C. Zhang. LAPO: Latent-variable advantage-weighted policy optimization for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022.

[33] D. Ki, J. Oh, S.-W. Shim, and B.-J. Lee. Prior-guided diffusion planning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025.

[34] A. Nair, A. Gupta, M. Dalal, and S. Levine. AWAC: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.

[35] Z. Wang, A. Novikov, K. Zolna, J. S. Merel, J. T. Springenberg, S. E. Reed, B. Shahriari, N. Siegel, C. Gulcehre, N. Heess, and N. de Freitas. Critic regularized regression. In *Advances in Neural Information Processing Systems*, 2020.

[36] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[37] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He. Mean flows for one-step generative modeling. In *Advances in Neural Information Processing Systems*, 2025. Oral.

[38] J. Sheng, Z. Wang, P. Li, and M. Liu. MP1: MeanFlow tames policy learning in 1-step for robotic manipulation. In *AAAI Conference on Artificial Intelligence*, 2026.

[39] G. Zhan, L. Tao, P. Wang, Y. Wang, Y. Chen, Y. Li, H. Li, M. Tomizuka, and S. E. Li. Mean flow policy with instantaneous velocity constraint for one-step action generation. In *International Conference on Learning Representations*, 2026. Oral.

[40] W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.

[41] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems*, 2018. doi:10.15607/RSS.2018.XIV.049.

[42] C. Bao, H. Xu, Y. Qin, and X. Wang. DexArt: Benchmarking generalizable dexterous manipulation with articulated objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[43] Y. Mu, T. Chen, Z. Chen, S. Peng, Z. Lan, Z. Gao, Z. Liang, Q. Yu, Y. Zou, M. Xu, L. Lin, Z. Xie, M. Ding, and P. Luo. RoboTwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[44] Q. Zhang, Z. Liu, H. Fan, G. Liu, B. Zeng, and S. Liu. FlowPolicy: Enabling fast and robust 3D flow-based policy via consistency flow matching for robot manipulation. In *AAAI Conference on Artificial Intelligence*, volume 39, pages 14754–14762, 2025.

Appendix

A One-Step Decoder Background

AWLS treats the one-step decoder G_θ as a frozen black box: the only property the rest of the paper relies on is that $G_\theta(\boldsymbol{o}, \boldsymbol{\epsilon})$ maps an observation $\boldsymbol{o}$ and a Gaussian latent $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ (of the same shape as the action chunk) to an executable action chunk in a single network evaluation. Both flow-based backbones we use, MeanFlow and FlowPolicy, provide this interface; we refer to the original papers for their training objectives.

A common one-step interface. Although MeanFlow and FlowPolicy differ in parameterization (interval-average velocity $\boldsymbol{u}_\theta$ vs. instantaneous velocity $\boldsymbol{v}_\theta$) and training signal (an exact JVP-based stop-gradient identity vs. a finite-difference EMA consistency objective), they yield the *same* interface for downstream use. In both cases the trained network collapses, at its boundary evaluation (the full interval $r = 0, t = 1$ for MeanFlow; $t = 1$ for FlowPolicy), to a single frozen map

$$G_\theta : (\boldsymbol{o}, \boldsymbol{\epsilon}) \longmapsto \boldsymbol{x}_0, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I), \tag{8}$$

that transforms an observation $\boldsymbol{o}$ and a Gaussian latent $\boldsymbol{\epsilon}$ (of the same shape as the action chunk) into an executable action chunk in one network evaluation. AWLS relies only on this interface and treats G_θ as a frozen black box; it is therefore agnostic to which backbone—MeanFlow or FlowPolicy—produced the decoder.

B Critic and Advantage Estimation

Advantages are estimated following IQL. A 2-ensemble of Q-functions yields a pessimistic estimate $Q_{\text{pess}}(\boldsymbol{o}, \boldsymbol{a}) = \min_k Q_{\psi_k}(\boldsymbol{o}, \boldsymbol{a})$. The value baseline is trained with expectile regression on this pessimistic estimate:

$$\mathcal{L}_V(\nu) = \mathbb{E}_{(\boldsymbol{o},\boldsymbol{a})\sim\mathcal{D}}\Big[\mathcal{L}_2^\tau\big(Q_{\text{pess}}(\boldsymbol{o}, \boldsymbol{a}) - V_\nu(\boldsymbol{o})\big)\Big], \qquad \tau > 0.5, \tag{9}$$

where $\mathcal{L}_2^\tau(u) = |\tau - \mathbb{1}\{u < 0\}|u^2$. The detached advantage used by AWLS is

$$A(\boldsymbol{o}, \boldsymbol{a}) = \text{sg}[Q_{\text{pess}}(\boldsymbol{o}, \boldsymbol{a}) - V_\nu(\boldsymbol{o})]\,, \tag{10}$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator. We use the goal-conditioned binary reward $r = \mathbb{1}[\,\text{task-success predicate at step } t\,]$: the reward is 1 on the transition that first satisfies the per-task success predicate already used by the simulator to score rollouts (the same predicate that defines the reported success rate—e.g. the object reaching its target pose/region for the DexArt and Adroit tasks, or the ordered stage being completed for RoboTwin), and 0 otherwise, with the episode terminating ($d = 1$) on success. Using the evaluation predicate as the reward keeps the offline advantage aligned with the metric we report. We train the Q-functions with Bellman regression—augmented with a conservative Q-learning penalty ($\alpha_{\text{CQL}} = 0.5$) that suppresses over-estimation on out-of-support noise—using target $y = r + \gamma(1 - d)\, V_{\bar{\nu}}(\boldsymbol{o}')$.

Does the critic rank clean demonstrations above noisy ones? The mixed-quality analysis in the main paper attributes AWLS's preference for cleaner demonstrated behavior to the critic assigning larger advantage weights to clean actions. We test this premise by measuring whether the offline advantage separates clean from noisy demonstrated actions. On each of the four $50/50$ clean/noisy mixed DexArt datasets we split the demonstrated pairs $(\boldsymbol{o}, \boldsymbol{a})$ into the clean subset and the Ornstein–Uhlenbeck–perturbed noisy subset (per-task perturbation magnitude σ), score every pair with the detached advantage A (Eq. 10) and its scale-normalized form $\tilde{A} \in [0, 1]$ (Eq. 6, the quantity entering the weight $w = \exp(\beta\tilde{A})$ at $\beta = 3$), and compare the two subsets in Table 4 and Figure 5.

Three observations support the claim. (i) On every task the critic assigns higher advantage to clean than to noisy actions, so clean actions receive 2.1–2.6$\times$ ($\approx 2.3\times$ on average) the weight of noisy

Table 4: **The IQL critic ranks clean demonstrations above noisy ones** (four $50/50$ clean/noisy mixed DexArt datasets, MeanFlow backbone; advantages detached). For each task we report the mean scale-normalized advantage $\tilde{A} \in [0, 1]$ (mean$\pm$std over three seeds) on the clean vs. OU-perturbed noisy subset, the induced weight ratio $w_{\text{clean}}/w_{\text{noisy}} = \exp\big(\beta(\tilde{\bar{A}}_{\text{clean}} - \tilde{\bar{A}}_{\text{noisy}})\big)$ at $\beta = 3$, the AUROC (empirical rank-order) of separating clean from noisy actions by advantage, and the Spearman correlation $\rho(A, R)$ between advantage and demonstrator return. Clean actions receive higher advantage on every task; the separation is strong and roughly uniform across tasks because each per-task σ is tuned to drive its noisy pool to a similar low quality ($< 20\%$ success), so σ reflects task sensitivity to noise, not the quality gap. Raw A is in return units (goal-conditioned, $\gamma = 0.99$); $\tilde{A}$ is its per-task min–max normalization, so the $[0, 1]$ scale is the per-task advantage range by construction. The bottom row is the per-task mean of each column.

Task	σ_{OU}	$\tilde{A}_{\text{clean}}$	$\tilde{A}_{\text{noisy}}$	$w_{\text{clean}}/w_{\text{noisy}}$	AUROC	$\rho(A, R)$
Laptop	0.4	0.66 ± 0.15	0.39 ± 0.17	2.26	0.89	0.47
Faucet	0.6	0.63 ± 0.15	0.39 ± 0.17	2.09	0.86	0.49
Toilet	0.6	0.66 ± 0.14	0.35 ± 0.17	2.55	0.92	0.56
Bucket	0.15	0.65 ± 0.15	0.40 ± 0.18	2.10	0.85	0.46
DexArt avg.	–	0.65	0.38	2.25	0.88	0.49

ones—all inside the bounded-weight regime $w \in [1, e^{\beta}]$ of Prop. 1, so the separation never collapses the effective sample size. (ii) Advantage is positively correlated with the demonstrator's return on every task ($\rho(A, R) > 0$), i.e. the critic ranks demonstrated actions by realized quality rather than arbitrarily. (iii) The separation is strong and roughly uniform across tasks (AUROC 0.85–0.92), as expected: although the per-task OU scale σ spans 0.15–0.6, it is tuned per task so that every noisy pool reaches a similar low quality (expert rollout success below 20%, by construction of the mixed-quality datasets), so the clean/noisy quality gap—and hence the critic's ability to separate the two—is comparable across tasks. The small σ for BUCKET reflects that task's higher *sensitivity* to action noise, not a milder quality gap. The aggregate weighting is consistent with the residual shaper outperforming the direct-interface variant on clean DexArt, where the direct variant is 2.5 points worse (Table 3(b)).

Two clarifications keep this evidence honest. The clean/noisy split is a provenance label that the critic never observes—it is trained only on the goal-conditioned success reward—so a high clean-vs-noisy AUROC is *not* circular: the critic *recovers* a demonstration-quality distinction it was never given. The correlation $\rho(A, R)$, in contrast, is partly definitional, since the reward is the deployment success predicate; a positive ρ therefore confirms that the advantage faithfully encodes the reported metric rather than establishing an independent notion of quality. The real-world study (Appendix I), whose within-success human demonstrations vary in execution quality, is intended to provide that independent axis, and the $50/50$ mix used here is the most balanced (and hardest-to-disentangle) setting, where clean and noisy modes are equally represented.

Do advantages separate quality differences within successful real-world demonstrations? The clean-vs-noisy separation above concerns synthetic corruption. To test whether the binary-reward critic also captures local quality variation inside successful human demonstrations, we reviewed every step of the 70 real-world Tube Insertion demonstrations (Appendix I), manually flagged problematic segments, and assigned each flagged step to exactly one category: *detour*, *hesitation*, or *jerk/abrupt motion*. Within each trajectory, steps whose advantage $A_t = A(\boldsymbol{o}_t, \boldsymbol{a}_t)$ falls below the trajectory median form the Low-A set (fixed tie-breaking). Table 5 reports, for each category, the share of its steps that fall in Low-A; a share above the 50% chance level indicates segments the critic identifies as lower quality. Detours (64.0%) and hesitation (71.3%) are strongly enriched, whereas jerk/abrupt motion stays at chance (50.7%): the binary-reward critic captures progress-related inefficiency well, while fine-grained smoothness is largely invisible to it.

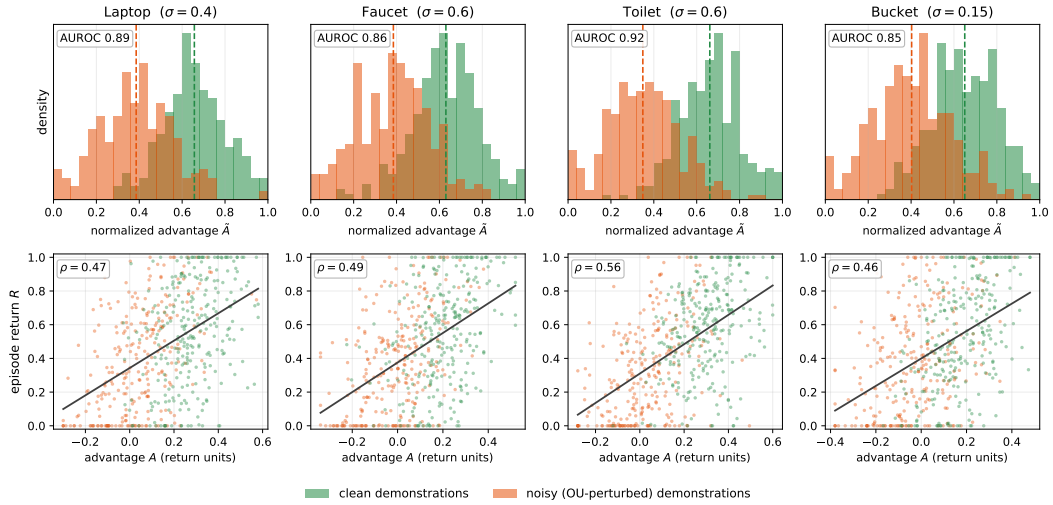


Figure 5: **The IQL critic ranks clean demonstrations above noisy ones** (four $50/50$ mixed DexArt datasets, MeanFlow backbone). **Top:** per-task histograms of the scale-normalized advantage $\tilde{A} \in [0, 1]$ on clean (green) vs. OU-perturbed noisy (orange) demonstrated actions; dashed lines mark the subset means. Clean actions carry systematically higher advantage on every task, and advantage separates the two subsets with empirical rank-order AUROC 0.85–0.92—even though clean/noisy provenance is a label the critic never observes during training. **Bottom:** the detached advantage A (return units) vs. the demonstrator's episode return R, with the linear trend; advantage is positively correlated with return on every task (Spearman $\rho = 0.46$–0.56). Summary statistics match Table 4; histograms and scatters pool all demonstrated pairs over the three seeds.

Table 5: Advantage–quality alignment on 70 real-world Tube Insertion demonstrations: share of manually flagged steps in each category that fall in the low-advantage half of their trajectory (50% = chance).

Category	Detour	Hesitation	Jerk/abrupt
In Low-A	$626/978$ ($\mathbf{64.0}\%$)	$345/484$ ($\mathbf{71.3}\%$)	$149/294$ (50.7%)

C Reconstruction Objective and Residual-Map Geometry

This appendix distinguishes the reconstruction objective used by AWLS from marginal action likelihood, then establishes a conditional result for the local geometry of the residual shaper. We evaluate retained action diversity separately rather than infer it from the residual form.

Reconstruction and likelihood. Write the composed policy as $f_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = G_\theta(\boldsymbol{o}, T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}))$. For an auxiliary Gaussian observation model $p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}, \boldsymbol{\epsilon}) = \mathcal{N}(\boldsymbol{a}; f_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}), \sigma^2 I)$ with fixed $\sigma^2 > 0$, the expected conditional negative log-likelihood is

$$\mathbb{E}_{\boldsymbol{\epsilon}}[-\log p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}, \boldsymbol{\epsilon})] = \frac{1}{2\sigma^2}\, \ell^{\phi}_{\mathrm{MSE}}(\boldsymbol{o}, \boldsymbol{a}) + \text{const.} \tag{11}$$

This is a conditional Gaussian interpretation of reconstruction; σ is an analytical device and AWLS adds no Gaussian action noise at deployment. For the auxiliary marginal $p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}) = \mathbb{E}_{\boldsymbol{\epsilon}}[p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}, \boldsymbol{\epsilon})]$, Jensen's inequality gives

$$-\log p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}) \le \mathbb{E}_{\boldsymbol{\epsilon}}[-\log p_\sigma(\boldsymbol{a} \mid \boldsymbol{o}, \boldsymbol{\epsilon})]\,. \tag{12}$$

Positive detached weights preserve this inequality after averaging over observations and demonstrated actions, but do not ensure that the bound is tight or that optimizing it matches a marginal-likelihood update.

Table 6: Noise-shaper structure and conditional geometric properties. The residual bounds require $\|J_\Delta\|_{\mathrm{op}} \le L_v < 1$; this condition is measured locally rather than enforced by AWLS.

	Gaussian	Direct NN	Residual
J_T	$\mathrm{diag}(\boldsymbol{\sigma}_\phi)$	J_{f_ϕ}	$I + J_\Delta$
Nonsingularity	$\min_i \sigma_i > 0$	Requires a condition	If $L_v < 1$
$\kappa_2(J_T)$	$\max_i \sigma_i / \min_i \sigma_i$	Unconstrained	$\le (1 + L_v)/(1 - L_v)$

The deterministic pushforward policy used at deployment has a different likelihood. If both maps are dimension-matched diffeomorphisms, let $\boldsymbol{\epsilon}_{\boldsymbol{a}} = f_\phi(\boldsymbol{o}, \cdot)^{-1}(\boldsymbol{a})$ and $\boldsymbol{z}_{\boldsymbol{a}} = T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}_{\boldsymbol{a}})$. The change-of-variables formula [40] gives

$$-\log \pi_\phi(\boldsymbol{a} \mid \boldsymbol{o}) = -\log p_{\boldsymbol{\epsilon}}(\boldsymbol{\epsilon}_{\boldsymbol{a}}) + h_G(\boldsymbol{o}, \boldsymbol{z}_{\boldsymbol{a}}) + h_T(\boldsymbol{o}, \boldsymbol{\epsilon}_{\boldsymbol{a}}), \tag{13}$$

where $p_{\boldsymbol{\epsilon}}$ is the standard Gaussian density, $h_G = \log|\det(\partial G_\theta/\partial \boldsymbol{z})|$, and $h_T = \log|\det(\partial T_\phi/\partial \boldsymbol{\epsilon})|$. These terms are evaluated at the demonstrated action's inverse latent, whereas reconstruction averages over independent latent draws. AWLS therefore uses an AWR-inspired supervised surrogate; we do not assert an MSE–marginal likelihood identity or a bound on their gradient discrepancy.

Conditional local geometry. For a fixed observation, the residual map $T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon} + \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$ has Jacobian $J_T = I + J_\Delta$, where $J_\Delta = \partial \Delta_\phi / \partial \boldsymbol{\epsilon}$. The following result controls the local deformation of this map.

Proposition 2 (Local conditioning of the residual map)**.** *Let $\Delta_\phi(\boldsymbol{o}, \cdot) : \mathbb{R}^d \to \mathbb{R}^d$ be continuously differentiable. At any state–noise pair satisfying $\|J_\Delta\|_{\mathrm{op}} \le L_v < 1$, the shaper Jacobian is nonsingular and obeys*

$$1 - L_v \le \sigma_{\min}(J_T) \le \sigma_{\max}(J_T) \le 1 + L_v, \qquad \kappa_2(J_T) \le \frac{1 + L_v}{1 - L_v}. \tag{14}$$

Its local log-determinant satisfies

$$d \log(1 - L_v) \le h_T \le d \log(1 + L_v). \tag{15}$$

Proof. For any vector $\boldsymbol{v}$, the triangle and reverse triangle inequalities give

$$(1 - L_v)\|\boldsymbol{v}\|_2 \le \|(I + J_\Delta)\boldsymbol{v}\|_2 \le (1 + L_v)\|\boldsymbol{v}\|_2.$$

Taking the infimum and supremum over unit vectors yields the singular-value bounds and hence the condition-number bound. Since $1 - L_v > 0$, J_T is nonsingular. The absolute determinant is the product of its singular values; taking logarithms gives Eq. (15). □

Remark 1 (Scope and measured evidence)**.** *The proposition is a pointwise geometric result. If its condition holds uniformly over all $\boldsymbol{\epsilon} \in \mathbb{R}^d$ for a fixed observation, then Δ_ϕ is L_v-Lipschitz and T_ϕ is globally invertible: for any target $\boldsymbol{z}$, the inverse is the unique fixed point of the contraction $\boldsymbol{\epsilon} \mapsto \boldsymbol{z} - \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$. AWLS does not enforce this uniform condition. Table 11 reports that $\|J_\Delta\|_{\mathrm{op}} < 1$ at every evaluated state–noise sample, supporting nonsingular local geometry at those points rather than certifying global invertibility.*

Local Jacobian control also does not bound residual displacement: a constant correction can have zero Jacobian and still shift the latent input substantially. We therefore measure displacement separately, alongside the Jacobian diagnostics. Neither measurement establishes a bound on the MSE–likelihood discrepancy or guarantees that generated actions stay within data support; action diversity and task performance are assessed empirically. Table 6 summarizes these conditional properties for the Gaussian, direct, and residual shaper structures, and Figure 6 compares the residual and direct shapers empirically on clean DexArt (the comparison behind Table 3(b)).

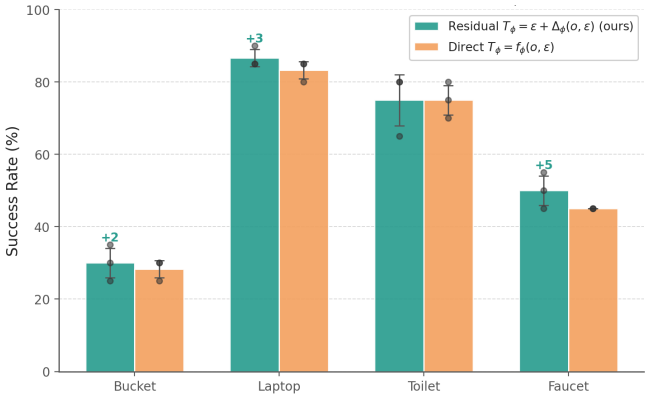


Figure 6: Residual vs. direct noise shaper on clean DexArt (ablation batch, mean±std over three seeds, best checkpoint). The residual parameterization $T_\phi = \epsilon + \Delta_\phi$ matches or exceeds the unconstrained direct map $T_\phi = f_\phi$ on every task. This comparison supports the residual parameterization empirically; its local geometric measurements are reported separately in Table 11.

Table 7: Latent diversity and success of noise-shaper variants on DexArt (two-seed matched reruns; shared frozen base per task and seed). D_{rel} is the mean pairwise action spread across latent samples relative to the BC backbone (1: base, 0: collapse).

	BC/Base	Direct	Gaussian	Gaussian + reg.	**Residual**
$D_{\text{rel}} \uparrow$	1.000	5.4×10^{-4}	5.2×10^{-5}	0.380	**0.457**
SR (%) $\uparrow$	55.2	59.8	59.5	60.2	**61.9**

Objective surrogate and latent diversity. Because the marginal action likelihood of the composed policy is not directly tractable, AWLS uses advantage-weighted reconstruction as an AWR-inspired surrogate with the conditional Gaussian interpretation above. Regressing independent noise draws toward demonstrated targets can reduce latent diversity by mapping different draws to similar actions. Table 7 reports the relative action diversity D_{rel}, the mean pairwise spread of decoded actions across latent samples at matched held-out states (one shared noise book), normalized by the spread of the BC backbone from the same run (1: base diversity, 0: collapse), together with success rate. All entries aggregate two-seed, newly retrained runs over the four DexArt tasks; for each task and seed the post-training variants share the same frozen base checkpoint. The residual shaper retains 0.457 of the base policy's action spread, more than any of the other tested shapers, and reaches the highest success rate among these variants. These results show partial retention of diversity rather than preservation of the original conditional action distribution. An unconstrained direct map and a Gaussian reparameterization collapse diversity almost completely; entropy regularization partly restores it at the cost of an additional objective and coefficient.

D Proof of Proposition 1 (Bounded Weight Concentration)

We restate the claim of Proposition 1: for a minibatch of size n the AWLS weights satisfy $w_{\max}/w_{\min} \le R$ with $R = e^{\beta}$, and the effective sample size obeys $\text{ESS} = (\sum_i w_i)^2 / \sum_i w_i^2 \ge 4nR/(1+R)^2$, which evaluates to $\text{ESS} \ge 46.3$ for $\beta = 3$ and $n = 256$.

Proof of Proposition 1. **Step 1 (weight ratio).** Each AWLS weight is $w_i = \exp(\beta\, \tilde{A}_i)$ with the per-batch normalized advantage $\tilde{A}_i \in [0, 1]$ from Eq. (6). Since $x \mapsto e^{\beta x}$ is increasing for $\beta > 0$, $w_i \in [e^0, e^{\beta}] = [1, e^{\beta}]$, hence $w_{\min} \ge 1$, $w_{\max} \le e^{\beta}$, and $w_{\max}/w_{\min} \le e^{\beta} =: R$. This holds for any minibatch and any critic state, because $\tilde{A}_i \in [0, 1]$ by construction independently of the raw advantage scale.

Step 2 (two-point worst case). ESS is invariant to a common positive rescaling, so set $w_i \in [1, R]$. Writing $S_1 = \sum_i w_i$, $S_2 = \sum_i w_i^2$ and viewing $\mathrm{ESS} = S_1^2/S_2$ as a function of a single coordinate $w_j \in [1, R]$,

$$\frac{\partial\,\mathrm{ESS}}{\partial w_j} = \frac{2S_1S_2 - 2w_jS_1^2}{S_2^2} = \frac{2S_1}{S_2^2}\,(S_2 - w_jS_1),$$

whose sign switches at most once, from $+$ to $-$, as w_j increases (because $S_2 - w_jS_1$ is decreasing in w_j). The single-coordinate restriction is hence unimodal with an interior maximum, so over the box $w_j \in [1, R]$ its *minimum* is attained at an endpoint $w_j \in \{1, R\}$. Applying this coordinate-wise, the minimizer is a two-point configuration: k weights equal to R and $n - k$ equal to 1.

Step 3 (optimal split). With $p = k/n$ the fraction at R, $S_1 = n(1 + p(R - 1))$ and $S_2 = n(1 + p(R^2 - 1))$, so (relaxing p to $[0, 1]$)

$$\mathrm{ESS}(p) = n\,\frac{(1 + p(R-1))^2}{1 + p(R^2 - 1)}.$$

Setting the derivative to zero and clearing the positive denominator gives $(R+1)\,(1+p(R-1)) = 2\,(1 + p(R^2 - 1))$. Using $R^2 - 1 = (R - 1)(R + 1)$ this reduces to $(R - 1) = p(R - 1)(R + 1)$, i.e. $p^* = 1/(R + 1)$ (the endpoints $p \in \{0, 1\}$ give $\mathrm{ESS} = n$, so p^* is the interior minimum).

Step 4 (evaluating the bound). At $p^* = 1/(R+1)$, $1+p^*(R-1) = 2R/(R+1)$ and $1+p^*(R^2 - 1) = R$, so

$$\min_p \mathrm{ESS}(p) = n\,\frac{(2R/(R+1))^2}{R} = \frac{4nR}{(1+R)^2}.$$

Since the continuous relaxation lower-bounds the integer-constrained minimum, every admissible weight configuration satisfies $\mathrm{ESS} \geq 4nR/(1 + R)^2$.

Step 5 (numerical instance). For $\beta = 3$, $R = e^3 = 20.0855\ldots$ and $(1 + R)^2 = 444.60\ldots$, so with $n = 256$

$$\mathrm{ESS} \geq \frac{4 \cdot 256 \cdot 20.0855}{444.60} = 46.26\ldots\ (\approx 46.3),$$

i.e. even the worst-case two-point batch retains at least ≈ 46 effective samples out of 256, independent of the critic state. □

Remark 2 (Tightness and scope)**.** *The bound is* tight in the worst case*: $4nR/(1 + R)^2$ is the largest critic-independent floor possible, attained (in the continuous relaxation of Steps 2–4) by the two-point split with a fraction $p^* = 1/(R + 1)$ of weights at $w = R$. It is, however,* loose for realized data*: during training the normalized advantages $\tilde{A}$ spread across $[0, 1]$ rather than concentrating at the two extremes, so the realized per-batch ESS sits well above the floor and the adversarial two-point batch essentially never occurs. The guarantee is also* per minibatch*: the floor $4nR/(1 + R)^2$ holds for every batch and every gradient step—the scale that governs the gradient-estimator variance—for the batch size n used at that step, rather than for a single global, dataset-level ESS. Any normalization that maps advantages into $[0, 1]$, whether per-batch or with a fixed global range, inherits this floor: it is the bound $\tilde{A} \in [0, 1]$, not the choice of range, that caps $w \leq e^\beta$. What per-batch normalization adds over a fixed global range is* adaptivity*—the effective temperature $\beta_t = \beta/(\Delta_t + \varepsilon)$ tracks the within-batch advantage spread $\Delta_t = A^{\mathcal{B}}_{\max} - A^{\mathcal{B}}_{\min}$, so the weighting stays informative as the critic, and hence the advantage scale, evolves during training instead of collapsing toward uniform on batches that occupy a narrow sub-interval of a stale global range. The floor is forfeited only by* unnormalized *advantages: with a fixed temperature and an unbounded raw advantage range, the within-batch weight ratio $e^{\beta(A_{\max} - A_{\min})}$ is unbounded and an individual batch's ESS can collapse toward* 1.

E Checkpoint Selection Diagnostics

Metric definitions. Best-of-M reconstruction error draws M latents per validation pair and reports the minimum action-space error:

$$L_{\mathrm{BoM}}(\theta_k) = \mathbb{E}_{(\boldsymbol{o},\boldsymbol{a})\sim\mathcal{D}_{\mathrm{val}},\ \boldsymbol{z}_{1:M}\sim\mathcal{N}(0,I)}\left[\min_{j=1,\ldots,M} \left\|G_{\theta_k}(\boldsymbol{o}, \boldsymbol{z}_j) - \boldsymbol{a}\right\|^2\right]. \tag{16}$$

This is appropriate because the decoder is stochastic: we require that the validation action remain *reachable* under the latent-noise prior. Directional sensitivity probes the decoder Jacobian via finite differences:

$$S_J(\theta_k) = \mathbb{E}_{\boldsymbol{o}\sim\mathcal{D}_{\text{val}},\ \boldsymbol{z}\sim\mathcal{N}(0,I),\ \boldsymbol{u}\sim\text{Unif}(\mathbb{S}^{d-1})}\left[\frac{\left\|G_{\theta_k}(\boldsymbol{o},\boldsymbol{z}+h\boldsymbol{u}) - G_{\theta_k}(\boldsymbol{o},\boldsymbol{z})\right\|}{h}\right]. \tag{17}$$

For small h, this estimates the average directional magnitude $\|\partial G_{\theta_k}/\partial \boldsymbol{z}\ \boldsymbol{u}\|$ and helps detect checkpoints whose decoder has become locally insensitive to latent noise. When S_J is flat across $\mathcal{C}_\delta$, the selection rule reduces to choosing by BC quality, indicating that checkpoint choice is unlikely to affect the latent-steering mechanism.

Checkpoint diagnostics. Over a full 3000-epoch DexArt behavior-cloning run (checkpoints saved and diagnosed every 50 epochs), behavior-cloning success rises and then plateaus, whereas decoder sensitivity S_J peaks at a task-dependent epoch and then declines as continued training collapses the noise-to-action response; post-training success tracks S_J rather than BC success and degrades when AWLS is applied to over-trained, low-sensitivity bases. Among checkpoints whose L_{BoM} is within tolerance ($1.2\times$ the final-epoch loss), the criterion therefore selects the highest-S_J checkpoint. The per-task validation of the rule—selection regret against a full oracle checkpoint sweep and the S_J/success rank correlation—is reported in Table 8.

Does S_J predict the best checkpoint? (validation protocol). The diagnostics above specify *how* the rule selects a checkpoint; we now verify that the rollout-free criterion actually recovers the checkpoint that yields the best post-training policy. We make this precise with a one-time *oracle sweep*, run once on a held-out validation suite and never needed at deployment. We run the full AWLS post-training at every in-tolerance checkpoint $\theta_k \in \mathcal{C}_\delta$ and measure its rollout success $\text{SR}_{\text{AWLS}}(\theta_k)$, defining the oracle checkpoint $k^\star = \arg\max_k \text{SR}_{\text{AWLS}}(\theta_k)$. This SR-defined oracle is used only as a verification reference. When applying AWLS, the frozen base is selected by the offline S_J criterion within the L_{BoM} tolerance; rollout success rates are not used to make that selection. Two quantities then test the rollout-free rule, whose offline choice we denote $\hat{k}$ (the highest-S_J checkpoint within the L_{BoM} tolerance):

- **Selection regret** $\Delta = \text{SR}_{\text{AWLS}}(\theta_{k^\star}) - \text{SR}_{\text{AWLS}}(\theta_{\hat{k}})$. Small Δ means the offline criterion recovers a near-oracle base without any per-checkpoint rollouts.
- **Rank agreement**: the Spearman correlation $\rho\big(S_J(\theta_k),\ \text{SR}_{\text{AWLS}}(\theta_k)\big)$ over $k \in \mathcal{C}_\delta$. Positive ρ means decoder sensitivity is genuinely *predictive* of post-training quality across the sweep, not merely coincident with it at the peak.

Reporting Δ and ρ per task (means over the three seeds) gives the across-task validation. We regard the rule as validated when Δ falls within evaluation noise (a few success-rate points) and $\rho > 0$ on tasks with a responsive decoder. The flat-S_J case is informative rather than a failure: on a task whose sensitivity barely varies (e.g. BUCKET) ρ may be near zero, but there Δ is also small because every in-tolerance checkpoint steers comparably—so an uninformative sensitivity profile leaves the rule harmless rather than misleading. We report this measured oracle sweep (per checkpoint $\times$ seed) in Table 8 on the four DexArt tasks: the S_J-selected base incurs only 0.9-point average selection regret against the oracle (a few points at most per task), while selecting by BC quality leaves $+8.2$ on the table, and $\rho(S_J, \text{SR}_{\text{AWLS}})$ is strongly positive on responsive-decoder tasks (mean 0.78).

Why decoder sensitivity decays. The decay of S_J follows directly from behavior cloning sharpening the conditional action distribution. Early in training the flow decoder still maps a broad region of the latent-noise prior onto a spread of plausible actions, so a latent perturbation $\boldsymbol{z}+h\boldsymbol{u}$ produces a sizeable change in the decoded action and S_J is large. As BC proceeds, the decoder concentrates the mass it places on each observation onto the demonstrated action mode, driving the conditional map toward a near-deterministic function of the observation; the latent Jacobian $\partial G_\theta/\partial \boldsymbol{z}$ correspondingly shrinks and S_J falls. This is the same property that makes latent steering both attractive

Table 8: **Measured checkpoint-selection validation (oracle sweep)** on the four DexArt tasks (success rate %). All entries are means over three training seeds (20 evaluation episodes per task per seed). The S_J-selected column is the deployed MF-AWLS model and reproduces Table 16 exactly, including its per-task $\pm$ (standard deviation over the three seeds); BC-best and oracle are reported as seed means. For each task we run full AWLS post-training at *every* in-tolerance checkpoint and take the oracle $k^\star = \arg\max_k \mathrm{SR}_{\mathrm{AWLS}}$, the S_J-selected checkpoint $\hat{k}$ (Eq. 17, highest S_J within the L_{BoM} tolerance), and the BC-best checkpoint. $\Delta_{\mathrm{reg}} = \mathrm{SR}(k^\star) - \mathrm{SR}(\hat{k})$ is the selection regret against the oracle; $\Delta_{\mathrm{BC}} = \mathrm{SR}(\hat{k}) - \mathrm{SR}(\text{BC-best})$ is the gain over BC-quality selection; ρ is the Spearman correlation between S_J and AWLS success across the sweep. Small Δ_{reg} on every task means the rollout-free rule recovers a near-oracle base; $\Delta_{\mathrm{BC}} > 0$ reproduces the main-paper ablation (Table 3(c)); the S_J-selected checkpoint is the deployed MF-AWLS model, so its per-task results match the main run (Table 16). The DexArt-average $\rho = 0.63$ here includes the flat-S_J BUCKET task ($\rho = 0.16$); over the three responsive-decoder tasks the mean is 0.78.

Task	BC-best SR	S_J-sel. SR (ours)	Oracle SR	Δ_{reg}	Δ_{BC}	$\rho(S_J, \mathrm{SR})$
Laptop	79.0	86.0 ± 1.4	87.0	1.0	+7.0	0.78
Faucet	40.0	48.0 ± 1.2	48.8	0.8	+8.0	0.72
Toilet	65.0	78.0 ± 0.7	79.5	1.5	+13.0	0.84
Bucket	32.0	36.8 ± 1.9	37.2	0.4	+4.8	0.16
DexArt avg.	54.0	62.2 ± 1.3	63.1	0.9	+8.2	0.63

and fragile: AWLS can only reshape behavior that the frozen decoder still exposes through its latent input, so post-training gains are largest at an intermediate checkpoint where the decoder remains responsive yet has already acquired competent behavior, and shrink once over-training has collapsed that degree of freedom.

Measured oracle-sweep validation. We now instantiate the protocol above with measured numbers, answering directly whether the rollout-free rule recovers a near-best AWLS checkpoint *across tasks*. This is the self-contained checkpoint-sweep run of the main-paper ablation, Table 3(c): on all four DexArt tasks we run full AWLS post-training at every in-tolerance checkpoint, identify the oracle $k^\star = \arg\max_k \mathrm{SR}_{\mathrm{AWLS}}(\theta_k)$, and compare it against the S_J-selected checkpoint $\hat{k}$ and the BC-best checkpoint, over three training seeds. Table 8 reports the per-task result. Three findings answer the question. (i) *Small regret*: the S_J-selected base is within $\Delta_{\mathrm{reg}} \leq 1.5$ of the oracle on every task (0.9 on the DexArt average), i.e. within the seed-to-seed variation of the success rates, so the offline rule recovers a near-oracle base without any per-checkpoint rollouts. (ii) *Large gain over the BC-quality heuristic*: choosing by S_J rather than by BC loss adds Δ_{BC} from +4.8 to +13.0 across the four tasks (mean +8.2; per task in Table 8), reproducing the main-paper ablation. (iii) *Predictive, not coincidental*: the Spearman correlation $\rho(S_J, \mathrm{SR}_{\mathrm{AWLS}})$ over the sweep is strongly positive on the three responsive-decoder tasks (mean 0.78) and near zero only on BUCKET, whose S_J is flat—there ρ is uninformative but $\Delta_{\mathrm{reg}} = 0.4$, so the rule is harmless rather than misleading. The S_J-selected checkpoint is the MF-AWLS model reported in the main paper, so its per-task DexArt results—and hence the 62.2 average—match Table 16 exactly. The oracle lies at most Δ_{reg} above it and BC-best Δ_{BC} below, so the small regret and positive rank agreement are independent of the operating point.

F Implementation Details

This section collects the architecture and hyperparameter settings used in our experiments, drawn from the released training configurations and the SNAFM-IQL post-training agent. On top of the frozen decoder, the AWLS post-training objective updates only the residual shaper ($\approx$ 1M parameters). The offline IQL advantage estimator—a critic ensemble ($\approx$ 0.58M) and a value network ($\approx$0.27M)—is trained separately and enters the objective only through detached advantage weights,

Algorithm 1 AWLS post-training on a frozen one-step decoder

Require: demonstrations $\mathcal{D}$; frozen-base candidates $\{\theta_k\}$; inverse temperature β; tolerance δ

(1) Rollout-free frozen-base selection (Appendix E)
for each candidate θ_k: compute $L_{\text{BoM}}(\theta_k)$, $S_J(\theta_k)$ (Eq. (16), (17))
$\mathcal{C}_\delta \leftarrow \{\theta_k : L_{\text{BoM}}(\theta_k) \le (1+\delta)\, L_{\text{BoM}}(\theta_{\text{final}})\}$
$\theta \leftarrow \arg\max_{\theta_k \in \mathcal{C}_\delta} S_J(\theta_k)$; **freeze** decoder G_θ

(2) Offline IQL critic (Appendix B)
train Q_ψ (2-ensemble, CQL) and V_ν (expectile τ) on $\mathcal{D}$
$A(\boldsymbol{o}, \boldsymbol{a}) \leftarrow \text{sg}[\min_k Q_{\psi_k}(\boldsymbol{o}, \boldsymbol{a}) - V_\nu(\boldsymbol{o})]$

(3) Advantage-weighted residual-shaper training
initialize $T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon} + \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$ near identity
for $t = 1, \dots, N$:
sample $\{(\boldsymbol{o}, \boldsymbol{a})\} \sim \mathcal{D}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$
$\tilde{A} \leftarrow (A - A^{\mathcal{B}}_{\min})/(A^{\mathcal{B}}_{\max} - A^{\mathcal{B}}_{\min} + \varepsilon)$; $w \leftarrow \exp(\beta\, \tilde{A})$ (Eq. (6))
$\boldsymbol{a}' \leftarrow G_\theta(\boldsymbol{o}, T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}))$ (frozen decoder)
$\phi \leftarrow \phi - \eta\, \nabla_\phi\, \mathbb{E}[\, w\, \|\boldsymbol{a}' - \boldsymbol{a}\|^2\,]$ (Eq. (4))
return residual noise shaper T_ϕ

with no gradient propagated into the shaper or the decoder. Algorithm 1 summarizes the end-to-end post-training procedure.

F.1 One-Step Decoder Backbones and Per-Benchmark Configuration

We instantiate AWLS on two one-step generative decoders: a MeanFlow decoder (MF-AWLS) and a FlowPolicy-style one-step decoder (FP-AWLS). Both consume a point-cloud observation through a DP3-style PointNet (XYZ) encoder (feature dimension 64, LayerNorm) together with a $[64, 64]$ proprioceptive state MLP, feeding a `ConditionalUnet1D` denoiser (channel dimensions $[512, 1024, 2048]$, step-embedding dimension 128, kernel size 5, 8 groups, FiLM conditioning; $\approx$255M parameters). Observations and actions are normalized to $[-1, 1]$. The MeanFlow decoder is trained with the JVP MeanFlow objective and an adaptive L_2 weighting ($\gamma = 0.5$), logit-normal (t, r) time sampling, classifier-free guidance weight $\omega = 2.0$, the AdamW optimizer (learning rate 10^{-4}, $\beta = (0.95, 0.999)$, weight decay 10^{-6}, batch size 128) with a linear-warmup (100–500 steps) then cosine-annealing schedule and gradient-norm clipping at 1.0, and an exponential moving average of the weights (decay 0.995); both decoders are frozen during AWLS post-training. The latent-noise dimension is $d =$ horizon $\times$ action dim. The per-benchmark observation and decoder configuration—shared by the DP3 baseline and the one-step backbones—is given in Table 9; optimizer settings common to all backbones appear in Table 10. The "held-out episodes per checkpoint" row is the number of held-out demonstration episodes from which the validation observations for the rollout-free diagnostics S_J and L_{BoM} are drawn (Appendix E); the diagnostics are computed at every saved checkpoint, i.e. every 50 epochs, and involve no policy rollouts.

F.2 Residual Noise Shaper T_ϕ

The residual noise shaper realizes $T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon} + \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$, where Δ_ϕ is an observation-conditioned network whose output is initialized near zero. Residual displacement and local Jacobian behavior are measured at the evaluated checkpoints (Table 11). The shaper is the only module updated during AWLS post-training; the decoder is frozen and the critic-derived advantage weights are detached. Table 10 lists its configuration.

Residual scale and measured residual-map statistics. Two shaper details warrant explanation. First, the *residual scale* in Table 10 is fixed to 1.0: the shaper applies the full additive correction Δ_ϕ without down-weighting. Orthogonal, gain-0.1 output initialization starts the map near the identity; no explicit small residual scale or Jacobian-norm constraint is imposed during training. The resulting displacement and local geometry are assessed at the evaluated checkpoints below. Second, the

Table 9: Per-benchmark observation and one-step decoder configuration. All benchmarks use point-cloud observations through a shared DP3-style encoder.

	Adroit	DexArt	RoboTwin
Observation modality	point cloud	point cloud	point cloud
Point-cloud points	512	1024	1024
Encoder feature dim	64	64	192
Prediction horizon	16	16	8
Observation steps	2	2	3
Action steps	8	8	6
Demonstrations per task	10	100	50
BC training length	3000 epochs	3000 epochs	1000 epochs
Held-out episodes per checkpoint (S_J, L_{BoM})	20	20	10
Random seeds	3	3	3

Table 10: Residual noise shaper T_ϕ (Δ_ϕ) architecture and AWLS post-training configuration.

Architecture	
Network Δ_ϕ type	MLP, $\boldsymbol{z} = \boldsymbol{\epsilon} + \text{scale} \cdot \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$
Hidden dimensions	$[512, 512, 512, 512]$
Activation / normalization	GELU / LayerNorm
Conditioning input	observation encoding, latent noise $\boldsymbol{\epsilon}$
Residual scale	1.0
Output initialization	orthogonal (gain 0.1), near-identity start
Output	residual correction Δ_ϕ
Trainable parameters	$\approx$ 1M
AWLS post-training	
Post-training steps	8000
Optimizer	Adam
Learning rate	3×10^{-4}
Weight decay	0
Gradient clipping	1.0 (max norm)
Batch size n	256
Random seeds	3

evaluated AWLS checkpoints apply the residual map $T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon}) = \boldsymbol{\epsilon} + \Delta_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$ directly to the Gaussian noise and pass $\boldsymbol{z} = T_\phi(\boldsymbol{o}, \boldsymbol{\epsilon})$ to the frozen decoder; *no sphere projection* is applied. (An earlier version of this appendix described a projection onto the sphere of radius $\sqrt{d}$, carried over from a previous method variant; it was not used by any reported model, and correcting the description changes no checkpoint or result.)

Since the Jacobian condition of Proposition 2 is not enforced during training, we measure it on the trained shaper. Table 11 summarizes a full-SVD evaluation of the exact residual-correction Jacobian $J_\Delta = \partial \Delta_\phi / \partial \boldsymbol{\epsilon}$ at 3,072 state–noise samples from the evaluated AWLS checkpoints. The learned correction is modest ($\|\Delta_\phi\|_2 / \|\boldsymbol{\epsilon}\|_2 \approx 0.175$), $\|J_\Delta\|_2 < 1$ holds at every sample with $Q_{.95}[\|J_\Delta\|_2] = 0.065$, and the transport Jacobian $I + J_\Delta$ stays well away from singularity ($Q_{.05}[\sigma_{\min}(I + J_\Delta)] = 0.966$, median condition number 1.04). These are local measurements at the sampled points, not a global certificate.

F.3 AWLS Objective

The AWLS loss (Eq. (4)) weights the noise-averaged reconstruction MSE by $w(\boldsymbol{o}, \boldsymbol{a}) = \exp(\beta \tilde{A}(\boldsymbol{o}, \boldsymbol{a}))$, where $\tilde{A} \in [0, 1]$ is the per-batch scale-normalized advantage of Eq. (6). Because $\tilde{A} \in [0, 1]$, the weight is bounded to $w \in [1, e^\beta]$ at every step. We use $\beta = 3$ for all tasks without per-task retuning. Table 12 lists the objective hyperparameters.

Table 11: Measured residual-map statistics of the trained shaper at 3,072 evaluated state–noise samples (full SVD of the exact Jacobian $J_\Delta = \partial \Delta_\phi / \partial \boldsymbol{\epsilon}$).

Residual size	Residual Jacobian J_Δ			Transport Jacobian $I + J_\Delta$	
Rel. norm $\|\Delta_\phi\|_2/\|\boldsymbol{\epsilon}\|_2$	Median $\|J_\Delta\|_2$	$Q_{.95}$ $\|J_\Delta\|_2$	Samples with $\|J_\Delta\|_2 < 1$	$Q_{.05}$ $\sigma_{\min}(I + J_\Delta)$	Median cond_2
≈ 0.175	0.036	0.065	3,072/3,072	0.966	1.04

Table 12: AWLS advantage-weighting objective hyperparameters.

Inverse temperature β	3
Advantage normalization	per-batch min–max (Eq. (6))
Normalization constant ε	10^{-8}
Normalized advantage range $\tilde{A}$	$[0, 1]$
Per-sample weight w	$\exp(\beta \tilde{A})$ (mean-normalized)
Weight bounds	$[1,\ e^{\beta}] \approx [1,\ 20.09]$
Within-batch weight ratio bound R	$e^{\beta} \approx 20.09$
ESS lower bound ($\beta = 3$, $n = 256$)	≥ 46.3
Reconstruction-noise samples per step	1

Table 13: IQL critic architecture and training configuration.

Q-ensemble size	2
Pessimistic aggregation	$\min_k Q_{\psi_k}$
Value baseline	V_ν, expectile regression
Expectile τ	0.7
Discount γ	0.99
CQL penalty weight α_{CQL}	0.5
Reward	goal-conditioned binary
Target update	Polyak ($\bar{\nu}$)
Target update rate	0.005
Q / V hidden dimensions	$[256, 256, 256, 256]$
Optimizer	Adam
Learning rate (Q, V)	3×10^{-4}
Gradient clipping	1.0 (max norm)
Batch size	256
Training steps	8000

F.4 IQL Critic

Advantages are estimated offline with IQL (Appendix B). We use a 2-ensemble of Q-functions and take their minimum as a pessimistic estimate $Q_{\mathrm{pess}} = \min_k Q_{\psi_k}$, train the value baseline V_ν with expectile regression ($\tau > 0.5$), and form the detached advantage $A = \mathrm{sg}[Q_{\mathrm{pess}} - V_\nu]$. The Q-functions are trained by Bellman regression with target $y = r + \gamma(1 - d)\, V_{\bar{\nu}}(\boldsymbol{o}')$ under goal-conditioned binary rewards. Critic gradients are never propagated into the policy. Table 13 lists the critic configuration.

F.5 Checkpoint-Selection Diagnostics

Frozen-base checkpoints are selected with the two rollout-free diagnostics of Appendix E: best-of-M reconstruction error L_{BoM} (Eq. (16)) over M latent samples, and directional sensitivity S_J (Eq. (17)) under latent-noise perturbations of magnitude h. The selection retains checkpoints within a multiplicative tolerance δ of the final-checkpoint L_{BoM} and picks the highest-S_J survivor (Eq. (7)). Rollout SR is used only to verify the checkpoint-selection rule and is not an input to the AWLS selection decision. Table 14 lists the diagnostic hyperparameters.

Table 14: Checkpoint-selection diagnostic hyperparameters.

Latent samples per pair M (L_{BoM})	16
Perturbation magnitude h (S_J)	0.01
Probe directions per state (S_J)	50
Validation observations (S_J)	32
L_{BoM} tolerance δ	0.2
Checkpoint saving / diagnostic interval	every 50 epochs
Validation set	offline held-out $\mathcal{D}_{\text{val}}$
Perturbation direction $\boldsymbol{u}$	Unif($\mathbb{S}^{d-1}$)

Table 15: Inference cost per action chunk. NFE = neural function evaluations of the decoder; latency is the mean over 1000 calls on a single NVIDIA RTX 4090 (batch size 1, DexArt configuration: 1024 points, prediction horizon 16), encoder included; throughput is the implied closed-loop rate. One-step methods are $\approx 4.8\times$ faster than the 10-step diffusion methods, and AWLS matches its one-step backbone up to the ≈ 0.3 ms shaper.

Method	Generative backbone	NFE	Latency (ms)	Throughput (Hz)
DP3	diffusion (DDIM)	10	17.8	56
DSRL	DP3 + noise steering	10	17.8	56
FlowPolicy	one-step flow	1	3.4	294
FQL	one-step flow	1	3.4	294
MeanFlow	one-step	1	3.4	294
FP-AWLS (ours)	FlowPolicy $+ T_\phi$	1	3.7	270
MF-AWLS (ours)	MeanFlow $+ T_\phi$	1	3.7	270

F.6 Inference Cost

AWLS preserves the one-step inference of its frozen backbone: the residual shaper T_ϕ adds a single small MLP evaluation ($\approx$ 0.3 ms) before the one decoder call, so the deployed policy costs essentially the same as its one-step base and far less than multi-step diffusion. Table 15 reports the number of decoder function evaluations (NFE) and the measured wall-clock latency per action chunk. The one-step methods (FlowPolicy, FQL, MeanFlow, and both AWLS variants) run at a single NFE, whereas DP3 and DSRL require the full 10-step denoising loop; AWLS is therefore $\approx 4.8\times$ faster than DP3/DSRL while matching FlowPolicy/FQL, and the shaper adds under 10% over the bare MeanFlow decoder. The real-world RGB policies (ResNet-18 encoders, $\approx$ 267M-parameter decoder) are proportionally heavier but preserve the same $10:1$ one-step-vs-diffusion ratio.

G Full Simulation Results

Tables 16 and 17 report the per-task simulation results corresponding to the benchmark-level summary in the main paper: Table 16 covers the Adroit and DexArt benchmarks, and Table 17 covers RoboTwin together with the cross-task Overall average.

G.1 Isolating Advantage Weighting

Block (d) of the main-paper ablation (Table 3) is expanded in Table 18. All variants follow the main-text evaluation protocol on the four DexArt tasks (MeanFlow backbone, clean demonstrations) and are compared against the main-result MF-AWLS (62.2). "Selected" is the checkpoint chosen by the rollout-free criterion of Eq. (7), which serves as the shared frozen base for every post-training variant; because the criterion targets post-training suitability rather than raw success, it sits slightly below the final BC checkpoint (54.0 vs. 55.2), consistent with the diagnostics in Appendix E. *Uniform* trains the same shaper with all weights equal to one; *Shuffled* randomly permutes the advantages across samples, so the weights keep their marginal distribution but lose their association with the actions; *Adv-filter* keeps only the top-50% of demonstrations ranked by their mean advantage and trains the shaper on them with uniform weights. Uniform and shuffled reweighting remain at

Table 16: Full per-task success rates (%) on the Adroit and DexArt simulation benchmarks. Each entry is mean±std over three random seeds (20 evaluation episodes per task per seed); the Avg. column under each benchmark is the per-benchmark task average and matches the benchmark summary (Table 1). RoboTwin per-task results and the cross-benchmark Overall average are in Table 17. The Avg. (and Overall) ± values are the average of the per-task standard deviations.

Method	Adroit (3 tasks, 10 demos)				DexArt (4 tasks, 100 demos)				
	Hammer	Door	Pen	Avg.	Laptop	Faucet	Toilet	Bucket	Avg.
DP3	**100.0±0.0**	55.8±5.7	46.8±4.2	67.5±3.3	75.8±0.8	35.8±1.5	61.8±1.1	27.8±1.3	50.3±1.2
FlowPolicy	**100.0±0.0**	61.0±3.4	53.0±2.6	71.3±2.0	76.6±3.0	36.4±2.1	63.4±1.8	25.4±1.9	50.4±2.2
MeanFlow	**100.0±0.0**	62.2±7.1	54.2±5.4	72.1±4.2	80.0±2.2	40.2±2.5	68.6±1.8	29.8±2.2	54.7±2.2
DSRL	**100.0±0.0**	62.6±4.2	55.0±3.2	72.5±2.5	81.4±3.6	41.8±4.5	71.4±3.5	30.8±3.1	56.4±3.7
FQL	**100.0±0.0**	64.0±5.1	56.8±2.6	73.6±2.6	83.0±2.2	42.8±2.3	72.8±1.8	31.2±3.1	57.4±2.4
FP-AWLS	**100.0±0.0**	66.0±3.0	58.4±4.9	74.8±2.6	83.2±2.8	43.2±1.9	72.6±2.1	35.2±1.8	58.5±2.1
MF-AWLS	**100.0±0.0**	**69.2±5.8**	**59.2±4.8**	**76.1±3.5**	**86.0±1.4**	**48.0±1.2**	**78.0±0.7**	**36.8±1.9**	**62.2±1.3**

Table 17: Full per-task success rates (%) on the RoboTwin simulation benchmark. Each entry is mean±std over three random seeds (20 evaluation episodes per task per seed). Avg. is the RoboTwin task average and Overall is the cross-task aggregate over all 12 tasks (Adroit, DexArt, and RoboTwin); both match the benchmark summary (Table 1). Adroit and DexArt per-task results are in Table 16. The Avg. (and Overall) ± values are the average of the per-task standard deviations.

Method	RoboTwin (5 tasks, 50 demos)						Overall Avg.
	Div. Bottles	Dual Bot. H	Empty Cup	Shoe Place	Beat Hammer	Avg.	
DP3	32.8±4.0	38.8±1.8	59.8±1.3	38.0±8.0	55.8±3.9	45.0±3.8	52.4±2.8
FlowPolicy	47.6±5.0	38.6±2.2	65.4±3.0	32.4±5.9	57.4±4.8	48.3±4.2	54.8±3.0
MeanFlow	52.4±8.2	42.2±2.6	69.8±1.6	39.6±5.0	62.2±5.4	53.2±4.6	58.4±3.7
DSRL	55.0±5.3	43.6±3.1	72.0±3.2	43.8±6.6	64.6±5.9	55.8±4.8	60.2±3.9
FQL	56.4±8.3	44.8±2.3	74.0±2.2	39.4±9.1	66.0±2.7	56.1±4.9	60.9±3.5
FP-AWLS	57.6±6.5	45.6±2.7	75.6±3.0	42.4±5.7	67.0±4.4	57.6±4.5	62.2±3.2
MF-AWLS	**63.4±7.7**	**50.8±2.9**	**78.6±2.2**	**46.0±2.5**	**69.2±3.1**	**61.6±3.7**	**65.4±2.9**

Table 18: Isolating advantage weighting on DexArt (MeanFlow; mean success rate over the four tasks, clean demonstrations). All post-training variants share the same selected frozen base; AWLS is the main-result MF-AWLS.

Variant	BC	Selected	Uniform	Shuffled	Adv-filter	**AWLS**
SR (%) ↑	55.2	54.0	55.9	55.3	60.2	**62.2**

BC level, hard filtering helps, and AWLS is best by retaining all demonstrations and reweighting them softly, which avoids the coverage loss of hard selection and its sensitivity to imperfect critic rankings.

G.2 Advantage-Temperature Sensitivity

Figure 7 sweeps the inverse temperature $\beta \in \{1, 3, 5, 10\}$ on the four DexArt tasks (MeanFlow backbone, clean demonstrations; DexArt-average success rate, mean over three seeds), comparing the scale-normalized advantage $\tilde{A} \in [0, 1]$ of Eq. (6) with the raw advantage $A(\boldsymbol{o}, \boldsymbol{a})$. The curves are an approximate reconstruction of the sweep figure in the submitted version of this paper: the values are read from that figure to within about half a success-rate point, and its ± 1 std bands over the three seeds are reproduced approximately. With normalization the success rate stays within about three points across the tested range (approximately 59.5, 61.5, 60.5, and 58.5), so $\beta = 3$ is used in all experiments without task-specific retuning. With the raw advantage the weights are unbounded (Proposition 1): performance is lower at every β (approximately 46.5, 48.5, 45.0, 37.0)

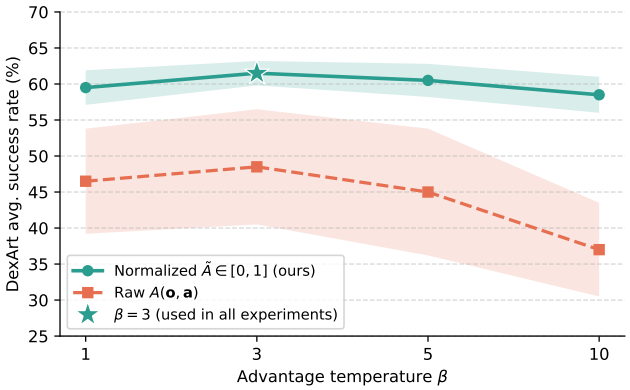


Figure 7: Advantage-temperature sweep on DexArt (MeanFlow, clean demonstrations; DexArt-average success rate, mean±std over three seeds). Approximate reconstruction of the submitted sweep figure: marker values are accurate to about ± 0.5 points and the shaded ± 1 std bands are reproduced approximately. The scale-normalized advantage (ours) is stable across β, whereas the raw advantage is lower and degrades as β grows. The star marks $\beta = 3$, used in all experiments.

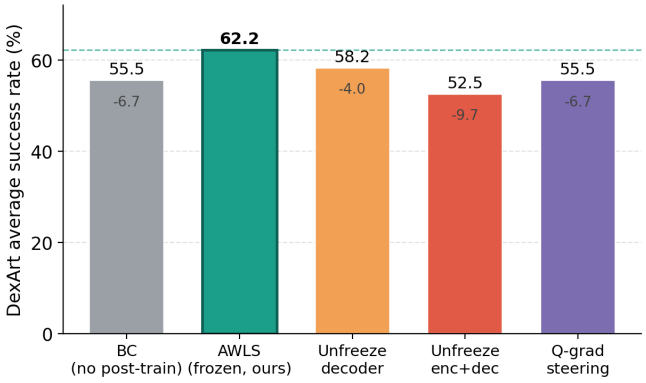


Figure 8: Decoder-adaptation ablation on DexArt (average success rate over the four tasks). Bars are anchored to MF-AWLS (62.2); the value below each bar is the ΔSR relative to MF-AWLS, matching the main-paper ablation (Table 3, block (a)). Freezing the decoder and steering the latent noise (AWLS, green) performs best, while Q-gradient steering matches the behavior-cloning prior.

and degrades as β grows. The sweep is a separate training run from the main-table MF-AWLS result (62.2), which is why its $\beta = 3$ value (about 61.5) differs slightly from Table 1.

G.3 Decoder-Adaptation Ablation

Figure 8 expands the decoder-adaptation block of the main-paper ablation (Table 3, block (a)) as a DexArt average. Keeping the one-step decoder frozen and steering only the latent noise (AWLS) is best; unfreezing the decoder, or jointly unfreezing the encoder and decoder, erodes the gain, and replacing the advantage-weighted supervised interface with a Q-gradient steering signal yields no improvement over the behavior-cloning prior.

H Mixed-Quality Demonstration Construction

For the DexArt mixed-quality study, we generate noisy demonstrations by executing the RL expert with temporally correlated OU action perturbations. At each control step, the perturbation evolves

as

$$\eta_{t+1} = \eta_t + \kappa(\mu - \eta_t)\Delta t + \sigma\sqrt{\Delta t}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0, I), \tag{18}$$

with zero mean $\mu = 0$, mean-reversion rate $\kappa = 0.15$, and control timestep $\Delta t = 0.1$ s (the 10 Hz control period). The perturbed action is clipped to the valid action range before execution. We tune the per-task noise scale σ (Bucket 0.15, Laptop 0.4, Toilet 0.6, Faucet 0.6; also annotated in the subplot titles of Figure 9) so that the expert rollout success rate falls from near 100% to below 20%, then use the resulting rollouts as the low-quality pool. Mixed datasets are constructed by replacing a chosen fraction of clean expert demonstrations with noisy expert-derived demonstrations while keeping the total number of demonstrations fixed.

Noisy-pool calibration and statistics. The per-task σ above is set by a coarse sweep so that the OU-perturbed expert reaches the target low-quality regime; the resulting noisy-pool rollout success rates are 18% (BUCKET), 16% (LAPTOP), 14% (TOILET), and 19% (FAUCET), all below the 20% ceiling. For each task we roll out the perturbed expert for 600 episodes across the three environment seeds (200 per seed) and keep only *usable* trajectories: time-limited rollouts are retained, but episodes that abort on a simulator fault or leave the valid workspace (3–6% per task) are filtered out, leaving ≥ 500 trajectories per task before sub-sampling to the requested mix fraction. The OU process and the clean/noisy assignment use fixed seeds $\{0, 1, 2\}$ (shared with the three training seeds), so the mixed datasets are reproducible. Both clean and noisy demonstrations come from the *same* RL expert and differ only in the injected action noise, so the two pools share state coverage and differ only in execution quality.

Limitations of the OU proxy. Ornstein–Uhlenbeck action perturbation is a controllable but imperfect stand-in for genuine human mixed-quality behavior: it injects temporally correlated, zero-mean noise around the expert action rather than the structured, goal-directed suboptimality of real teleoperators (hesitation, re-grasps, indirect paths, and pauses). It therefore captures the *noisy-execution* axis of mixed quality but not higher-level strategic suboptimality. The real-world experiments (Appendix I), whose within-success human demonstrations vary in speed, contact timing, and final alignment, complement this synthetic study along that second axis.

Main-paper Figure 3 reports the DexArt-averaged robustness curve and the action-trajectory PCA overview for this study; here we add the per-task decomposition (Figure 9).

Trajectory-PCA methodology. The action-trajectory PCA (main-paper Figure 3, right) flattens every demonstrated action chunk into a (horizon $\times$ action-dim) vector and fits a 2-component PCA *on the demonstration actions only* (clean and noisy demos pooled), so the projection is defined independently of any policy's rollouts; the same fitted basis is then applied to 200 generated rollouts per task for every method. PCA fitting uses fixed seeds $\{0, 1, 2\}$, so the embedding is reproducible.

I Real-World Experiment Details

This section expands the real-world setup of Section 4.3. All real-world policies use the MF-AWLS variant: AWLS freezes the MeanFlow behavior-cloning decoder and trains only the residual noise shaper T_ϕ. Consistent with the offline post-training setting of AWLS, no real-world rollouts are used at any point during post-training; the residual noise shaper and the offline critic are trained entirely on the teleoperated demonstrations, and the robot is used only for the final evaluation rollouts.

Hardware and control. The single-arm tasks use a Universal Robots UR5e manipulator and the bimanual tasks use two Universal Robots UR3e manipulators. Perception combines one third-view Intel RealSense D455 camera with one wrist-mounted Intel RealSense D405 camera per arm. Unlike the simulation benchmarks, which consume point clouds, the real-world policy operates directly on *RGB* images: a third-view stream (`img`) and a wrist stream (`rgb_hand`), each 256×256 and center-cropped to 224×224. Policies run in closed loop at 10 Hz.

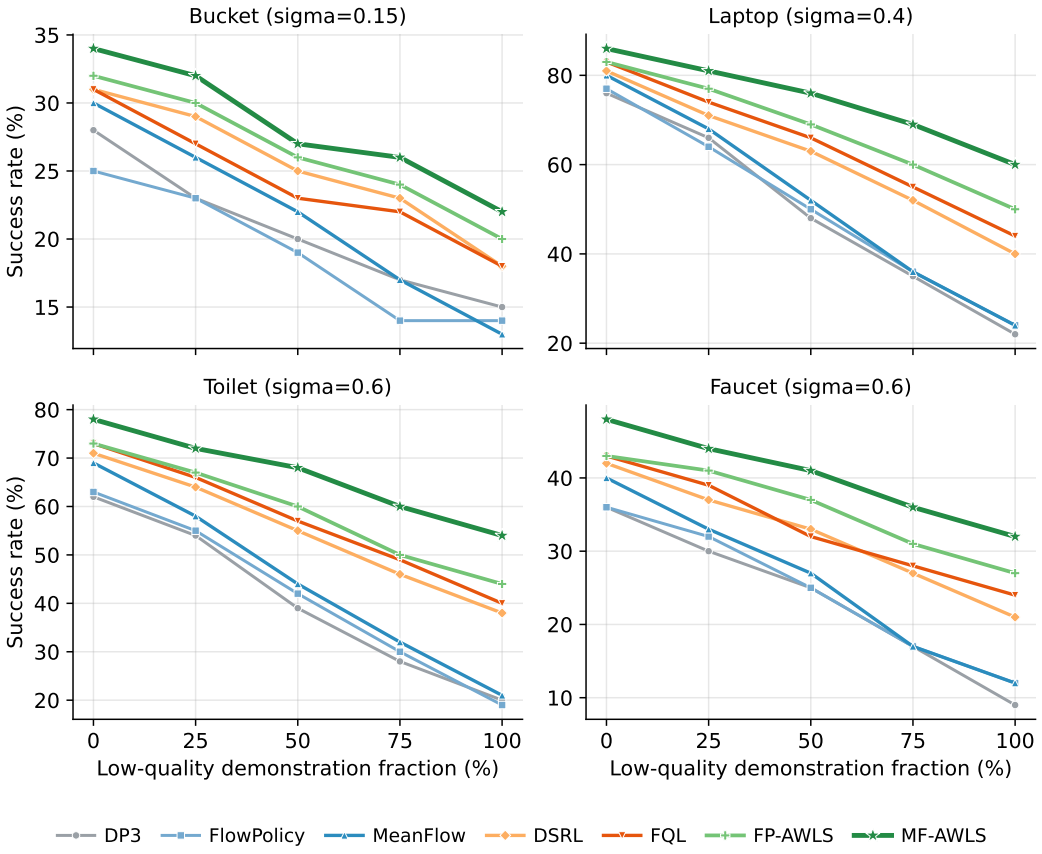


Figure 9: **Per-task breakdown of the mixed-quality robustness study.** Post-adaptation success rate (%) versus the fraction of low-quality demonstrations in the training mix, shown separately for each of the four DexArt tasks. This is the per-task decomposition of the DexArt-averaged curves reported in main-paper Figure 3; each subplot title lists the task's OU noise scale σ used to corrupt the low-quality demonstrations (larger σ = noisier). Across every task and at every contamination level, the AWLS variants (green) degrade most gracefully, with MF-AWLS (thickest, dark green) retaining the highest success rate even at 100% low-quality data, whereas the behavior-cloning backbones (gray/blue) fall off fastest. The Q-Max baselines (orange) lie in between. The consistent green-over-rest ordering within each panel shows the averaged result is not driven by any single task.

Observation and action spaces. Each RGB stream is encoded by an independent ImageNet-pretrained ResNet-18 backbone (global average pooling followed by a linear projection to a 128-dimensional feature); the two camera features are concatenated with a 64-dimensional embedding of the 7-dimensional proprioceptive state, giving a $2 \times 128 + 64 = 320$-dimensional observation feature. The action space is the 7-dimensional end-effector command (pose and gripper) for single-arm tasks. The frozen MeanFlow decoder predicts an action chunk of horizon 4 from 2 observation steps and executes the 4 predicted steps per inference. The encoder (≈ 22.5M parameters) and the MeanFlow `ConditionalUnet1D` decoder (down dimensions $[512, 1024, 2048]$, ≈ 267M parameters) total ≈ 289M parameters.

Demonstrations and BC training. All methods are trained from the same offline teleoperated demonstrations: every real-world task is collected with 70 demonstrations teleoperated with a SpaceMouse, with commands expressed in the robot base frame.[1] The MeanFlow decoder is behavior-

[1] All five tasks (Drawer-Cube, Tube Insertion, Screw Scooping, Gear Packing, and Ball Weighing) use the same 70 demonstrations per task and the identical recipe (encoder, decoder, training schedule, and evaluation protocol); all reported success rates are computed from completed evaluation rollouts on all five tasks.

cloned for 3000 epochs with AdamW (learning rate 10^{-4}, $\beta = (0.95, 0.999)$, weight decay 10^{-6}), batch size 128, a cosine schedule with a 500-step warmup, gradient-norm clipping at 1.0, and an EMA of the weights; checkpoints and evaluations are taken every 200 epochs. Even when successful, these demonstrations vary in speed, contact timing, pauses, corrections, and final alignment, producing the within-success mixed-quality conditions that advantage weighting is designed to exploit.

Post-training on real data. For each task we behavior-clone the MeanFlow decoder, select the frozen base checkpoint with the rollout-free S_J criterion of Eq. (7), train the offline IQL critic on the same demonstrations, and run AWLS on top of the 320-dimensional ResNet-18 observation features using the residual shaper and critic of Appendix F with detached, scale-normalized advantage weights ($\beta = 3$). Critic gradients are never backpropagated into the decoder, and no real rollouts are collected during this stage.

Baseline parity. FQL and DSRL are matched to MF-AWLS in every shared component so that the real-world comparison isolates the post-training method, not the representation. All four policies (frozen MeanFlow base, FQL, DSRL, MF-AWLS) use the identical perception stack—the two ImageNet-pretrained ResNet-18 encoders and the 320-dimensional observation feature described above—and are trained from the same 70 teleoperated demonstrations per task. FQL and DSRL reuse the same offline IQL critic (architecture and hyperparameters of Appendix F.4) trained on those demonstrations, are checkpoint-selected by the same rollout-free S_J rule (Eq. (7)), and receive the same post-training compute and hyperparameter-tuning budget—a single shared sweep over learning rate and steering strength with an equal number of gradient updates. The methods differ only in the generative decoder and steering module each one intrinsically requires: DSRL steers a 10-step diffusion decoder, whereas FQL and MF-AWLS use a one-step decoder of matched size (Table 15); none of the baselines uses real rollouts during post-training, exactly as for AWLS.

Evaluation protocol. Per-task arms, episode caps, and ordered-stage counts are listed in Table 19. Each policy is evaluated for 20 hardware rollouts per task from randomized but feasible initial configurations. An episode is capped at 300 control steps (30 s at 10 Hz), and a rollout counts as successful only if *every* ordered task stage is completed without human intervention. The per-task stages are: **Drawer-Cube**—open drawer, place cube, close drawer (3 stages); **Tube Insertion**—remove jar, insert tube (2); **Screw Scooping**—scoop screws, pour into the target container (2); **Gear Packing**—open box, place gear, close box (3); **Ball Weighing**—press the scale button and place the ball on the scale (2).

Table 19: Per-task real-world configuration. Episode steps is the rollout cap (300 steps = 30 s at 10 Hz); #Stages is the number of ordered stages that must all be completed for a rollout to count as a success.

Task	Arms	Episode steps	#Stages
Drawer-Cube	Single	300	3
Tube Insertion	Single	300	2
Screw Scooping	Single	300	2
Gear Packing	Dual	300	3
Ball Weighing	Dual	300	2

Success rates with confidence intervals. Table 20 expands the main-paper success counts (Table 2) with Wilson score 95% confidence intervals computed directly from the 20 rollouts per task (and $n = 100$ in aggregate); no additional experiments are involved. At the per-task level the 20-rollout intervals are necessarily wide and overlap across methods, so individual-task differences are not individually significant and we read the real-world result at the aggregate level. There, MF-AWLS reaches $65/100$ ($[55.3, 73.6]\%$) and its interval is disjoint from the frozen MeanFlow base ($45/100$, $[35.6, 54.8]\%$). The marginal intervals of MF-AWLS and the strongest Q-maximization

Table 20: Real-world success rate (%) with Wilson score 95% confidence intervals, computed from the 20 hardware rollouts per task in Table 2 ($n = 100$ aggregate). Per-task intervals are wide and overlap. By a task-stratified Cochran–Mantel–Haenszel test with Holm correction for three comparisons, only the gain over the BC base (MeanFlow, $p = 0.004$) is significant; the gains over DSRL ($p = 0.031$) and FQL ($p = 0.042$) are nominally below 0.05 but do not survive correction, so we report them as suggestive and treat the aggregate comparison as descriptive. No new experiments are involved—these are the same rollout counts.

Method	Drawer	Tube	Screw	Gear	Ball	Agg. (%)
MeanFlow	60 [39, 78]	55 [34, 74]	65 [43, 82]	20 [8, 42]	25 [11, 47]	45 [36, 55]
FQL	65 [43, 82]	70 [48, 85]	75 [53, 89]	25 [11, 47]	20 [8, 42]	51 [41, 61]
DSRL	60 [39, 78]	75 [53, 89]	65 [43, 82]	30 [15, 52]	20 [8, 42]	50 [40, 60]
MF-AWLS (ours)	**75** [53, 89]	**85** [64, 95]	**80** [58, 92]	**45** [26, 66]	**40** [22, 61]	**65** [55, 74]

baselines (FQL $51/100$, $[41.3, 60.6]$; DSRL $50/100$, $[40.4, 59.6]$) overlap. With five tasks of 20 rollouts each, the appropriate analysis stratifies by task and corrects for the three comparisons rather than pooling into a single binomial. A Cochran–Mantel–Haenszel test stratified by task gives MF-AWLS vs. MeanFlow $p = 0.004$, vs. DSRL $p = 0.031$, and vs. FQL $p = 0.042$; after Holm correction for the three comparisons, only the improvement over the frozen behavior-cloning base (MeanFlow) is significant at $\alpha = 0.05$, while the gains over FQL and DSRL are nominally below 0.05 but *do not survive* the correction. We therefore treat the aggregate comparison as descriptive, rest the significance claim only on the gain over the BC base, and report the FQL/DSRL margins as suggestive; the per-task numbers are shown transparently, since $n = 20$ per task is underpowered for task-level claims.